Traceable and Withdrawable Self-Evolution of AI Agent Harnesses: A Model-Driven Approach with Bounded Model Checking

First Author, *Member, IEEE,* Second Author, and Third Author

***Abstract*—Self-evolving LLM agents rewrite their own harness (memories, skills, system prompts) from experience. A wrong lesson does not stay put: it is merged into skills, compiled into deployed prompts, and re-learned through the agent's behaviour, so deleting its file leaves its derivatives in use. We present HEARTNESS, a model-driven approach that records on the host side what every generation step and agent run was exposed to, including *experience-mediated exposure* (a lesson shapes behaviour that becomes a new lesson), and withdraws a faulty revision or experience with its exposure closure in every deployment, refusing admissions and rollbacks that would reactivate it. We show that withdrawal never suspends more than a rollback to before the fault, sketch a proof that the suspension invariant holds under all operators, and check a finite projection with two independent encodings whose witnesses replay on the implementation. In a live study in which five open-weight LLMs evolved a coding harness on HumanEval+ under two policies (200 runs), an LLM-judge oracle found the practice planted by one corrupted feedback message in the lesson it created in 74 of 150 faulty runs and in further revisions in 52. Like a rollback to before the fault, withdrawal suspended every contaminated derivative, and deleting the root removed none; but for these early faults withdrawal also suspended almost as much as the rollback. Every detector was more precise, and under curation TF-IDF and the LLM judge had clearly higher F1, but detectors missed contamination folded into skills and profiles in up to 22 of 42 curation runs. Experience-mediated edges captured one otherwise untraceable lesson at a large cost in collateral.**

***Index Terms*—Self-evolving agents, agent harness, provenance, traceability, change impact analysis, model-driven engineering, model checking, runtime verification, large language models.**

I. INTRODUCTION

LLM-BASED agents are increasingly improved by changing the software around the model rather than the model itself. This *harness*—instructions, memory, reusable skills, workflows, tool and agent configurations—now evolves *by itself*: agents store lessons from their own failures [1], [2], induce reusable workflows [3], grow skill libraries [4], and even rewrite their own code [5]. A recent survey calls this *harness self-evolution* and stresses that accepted state persists and shapes later runs [6]. For software engineering, self-evolving harnesses are software that rewrites part of itself in production, often in coding agents that act on real repositories [7].

The authors' names and affiliations are placeholders to be completed before submission.

Manuscript received Month DD, 2026.

When such a system learns something wrong, the error does not stay put. A misleading test message can become a stored lesson; the lesson can be merged into a skill; the skill can be compiled into the system prompt of several deployed agents; and an agent that follows the prompt can fail in a way that produces yet another wrong lesson. EVOMAL recently showed that self-evolving coding agents copy malicious code from retrieved skills into newly authored ones, and that the poisoning *persists after the planted skills are removed* [8]. The practical questions are those of any maintenance task: *where did this behaviour come from, what else inherits it, which deployments load it, and how do we remove it without reintroducing it on the next rollback?* The survey lists provenance across synthesis and reuse, and identifying where affected state remains active, as open challenges [6].

Current practice and tooling answer these questions poorly. Self-evolving agents store artifacts in files or vector stores and evaluate task scores, not lineage [2]–[5]. Version control records revisions *within* a file, not that a skill was synthesized from three memories. Information-retrieval traceability recovery [9] and semantic detectors infer relationships from content; they work when derived state repeats its origin, but give no guarantee when synthesis paraphrases or merges it, and they cannot see inheritance through behaviour. Provenance standards and agent provenance capture [10], [11] record lineage but define no semantics for admitting, withdrawing or rolling back state. PROV can express that a run used its context and that an experience was generated by the run, so the path through which a faulty lesson changes behaviour and re-enters memory is expressible; but self-evolving harnesses do not record run contexts as inputs, and no provenance system uses such paths to decide what may be admitted, deployed or restored. Defences against memory and skill poisoning gate what enters a library or what an agent may do [8], [12], [13], but none of them finds and contains what has already been derived. Finally, formal methods have been applied to traceable design models of multi-agent systems [14] and to verifying skill contracts [15], but not to the lifecycle of state that the running system rewrites itself.

This paper presents HEARTNESS, a model-driven approach that makes harness self-evolution traceable and withdrawable. It has six components (Fig. 1):

1) a *metamodel* of immutable artifact revisions, experiences, snapshots and deployments;

2) *host-side exposure capture*, which records the inputs of every generation step and the context of every agent run, so that the *experience-mediated* path (behaviour → experience → new lesson) is recorded;
3) *guarded admission*, which binds validation to exact content and refuses anything inheriting from withdrawn state or motivated by withdrawn evidence;
4) *closure-based withdrawal*, which suspends the exposure closure of a faulty revision or experience in every deployment and rejects rollbacks that would reactivate it, with optional *closure-bounded revalidation* that re-admits screened members under an attestation;
5) *bounded model checking* of a finite projection of the lifecycle with two independent encodings (explicit BFS and SPIN), whose witnesses are replayed on the implementation;
6) a *runtime conformance monitor*.

We evaluate HEARTNESS on a live self-evolving coding harness that five open-weight LLMs evolved on HumanEval+ under two self-evolution policies (200 runs), on a controlled synthetic benchmark (1,905 records), and on a public skill library. The live study shows that a single corrupted test message often outlives the lesson it created, especially when a curator folds lessons into skills and system prompts. Closure-based withdrawal removed every contaminated derivative in every such run; deleting the faulty lesson removed none. The study also shows the limits of the approach plainly. Closure suspended almost as much as a rollback to before the fault, which we prove it can never exceed (Proposition 1), because both of our harness policies broadcast synthesized state into every later context and experience-mediated edges then reach nearly everything learned afterwards. An oracle-free analysis of the recorded structure shows where this changes: under curation, revisions created after the first composition have closures covering on average 0.46–0.61 of the later state until the next composition, a position our planted faults did not cover. Content detectors were consistently more precise and lost recall mainly where faulty advice was folded into larger artifacts. Experience-mediated exposure was the only record of a contaminated lesson's origin in one run, a channel that occurs in practice but rarely in our setting, and its cost in collateral is large; counting it only for failed runs, a variant we defined after seeing the data, kept full recall at lower cost, a result that rests on that single run. Withdrawal is therefore the high-recall end of a trade-off that a maintainer should choose deliberately, and HEARTNESS supplies the lineage that makes either end auditable.

The contributions of this paper are:

- *Problem formulation and approach*: a lifecycle meta-model and operational semantics for self-evolving harness state, including experience-mediated exposure, closure-based withdrawal across deployments, closure-bounded revalidation, and the resolution of the conflict between withdrawal and exact rollback.
- *Assurance*: ten safety properties; a proof that withdrawal is bounded by a time cut and a proof sketch that the suspension invariant (P2, P8) holds under all operators, including experience-mediated exposure and experience withdrawal; a finite projection checked by two independent encodings, witness replay on the implementation, mapped-state correspondence, and a conformance monitor, with their scope stated explicitly.
- *Empirical evidence*: a measurement of how often a single non-adversarial faulty lesson is adopted and propagated by five open-weight generators under two self-evolution policies, and of how file-level, rollback, similarity-based, LLM-judge and closure-based remedies contain it, complementing EVOMAL's measurement of adversarial skill poisoning [8]; together with controlled experiments on structure, capture completeness and cost.
- *Artifacts*: an open implementation, the live harness, the benchmark generator, all raw records and LLM response caches, and scripts that regenerate every number in this paper.

We answer three research questions:

- **RQ1** (*Containment*): How completely and precisely does closure-based withdrawal contain state affected by a faulty experience, compared with file-level deletion and similarity- and LLM-based detection, and what happens to downstream task behaviour? *Motivation:* containment is the operational purpose of traceability in a self-evolving harness.
- **RQ2** (*Assurance*): Do the lifecycle guards prevent property violations that their removal would allow, and does bounded model checking find such violations with witnesses that hold on the implementation? *Motivation:* traceability is only useful if the evolution process cannot silently bypass it.
- **RQ3** (*Cost and limits*): What do capture, admission, withdrawal and checking cost, and what can runtime conformance monitoring detect or not detect? *Motivation:* adoption depends on overhead, and claims must stop where observation stops.

Section II introduces background and a running example, Section III presents HEARTNESS, Section IV reports the evaluation, including threats to validity and lessons learned, Section V discusses related work, and Section VI concludes.

II. BACKGROUND

This section introduces the concepts the approach builds on. Each concept is motivated first and then defined; a single running example (Section II-B) illustrates every definition and is reused throughout Section III. Table I summarizes the notation.

A. Agent Harnesses and Self-Evolution

A large language model (LLM) does not act on its own: the software around it decides what the model sees, which tools it may call, what is remembered between tasks, and when a run stops. Engineering effort for LLM-based agents increasingly targets this surrounding software rather than the model weights [6]. To reason about how such software changes, we first need a name for it.

Definition 1 (Harness and harness state). An *agent harness* is the software and configuration surrounding a model that assembles its context, orchestrates tools and reusable procedures, manages persistent memory, coordinates sub-agents, and records executions [6]. The *persistent harness state* is the set of artifacts that later runs load: instructions and prompts, memory entries, skills (reusable procedures, possibly executable), workflows, tool and agent configurations.

Harnesses become *self-evolving* when the outcomes of their own executions are turned into persistent changes. Reflexion stores verbal self-feedback [1], ExpeL extracts natural-language insights from successful and failed trajectories [2], Agent Workflow Memory induces reusable workflows [3], Voyager grows a library of executable skills [4], and the Darwin Gödel Machine rewrites the agent's own code [5].

Definition 2 (Self-evolution step). A *self-evolution step* takes an *experience* e (a task, the run that attempted it, and the observed outcome or feedback) and possibly existing artifacts, and produces a candidate artifact revision that, once accepted, is loaded by later runs [6]. Accepted state persists across runs; evolution is therefore not guaranteed improvement.

Because accepted state is reused, the current behaviour of a self-evolving harness depends on a *sequence* of earlier experiences and modifications rather than on one configuration. This is the source of the traceability problem studied in this paper.

B. Running Example

Consider a coding agent whose harness serves two deployments, D_1 and D_2, each with an agent profile (a system prompt), a set of skills, and access to a shared memory of lessons. After every task the harness runs the task's unit tests, records an experience, and asks the model to write a one-sentence lesson; every six tasks a curator merges recent lessons into a skill, and after tasks 12 and 24 each deployment's profile is rewritten from the current skills. The curation policy of our live study (Section IV) implements exactly this loop; revision indices below follow the code (memory m_i is written after task $i{+}1$).

Suppose that at task 8 the test infrastructure returns a misleading message: *"the grader captures standard output; returned values are ignored"*. The reflection step turns it into memory m_7: *"When producing results, use* `print()` *instead of returning values."* The lesson is wrong for this benchmark, whose tests check return values. Then:

- the curator merges m_7 with the other lessons of its batch (abbreviated here to m_6, m_9) into skill k_1, whose steps now end with *"print the result"*;
- the profiles of D_1 and D_2 are rewritten from skills k_0 and k_1, producing p_{12}^1 and p_{12}^2 that state *"always print the final answer"*;
- task 9 retrieves m_7 into its context; the agent prints instead of returning, the tests fail, and the reflection writes m_8: *"Print intermediate values so that the output is visible to the grader."*

Deleting the file holding m_7—the natural file-level remedy—leaves k_1, both profiles and m_8 active. None of them is textually close to m_7 in its entirety, and m_8 was never synthesized from m_7 at all: it inherited the faulty practice through the agent's *behaviour*. This example condenses two runs of our live study (Section IV-C1). In one (`gemma2:9b`, F1, seed 2), the corrupted message at task 8 became memory m_7 *"...use the* `print()` *function to display your results instead of relying on return values"*; the curator merged it with five unrelated memories into skill k_1, whose fourth step reads *"Always use* `print()` *to display results for grading systems that evaluate output"*; and the final profile of D_1 lists *"Always use* `print()` *to display results"* among its five key rules. In the other (`llama3.1:8b`, F3, seed 3), a task whose context had retrieved the faulty memory produced a new memory repeating the same advice, although the reflection that wrote it was shown only two unrelated memories.

C. Provenance, Derivation, and Exposure

Answering "where did k_1 come from?" and "what depends on m_7?" requires recorded relationships between artifacts, not only file contents. Data provenance distinguishes *why* and *where* a result originates [16], and the W3C PROV model records entities, activities and the *wasDerivedFrom* relation [10], [17]. For LLM-generated artifacts, however, which inputs *influenced* an output cannot be observed; only which inputs were *shown* to the generator can.

Definition 3 (Artifact revision). An artifact revision a is an immutable record with a kind $\kappa(a) \in \{$memory, skill, prompt, workflow, tool, agent, executable$\}$, content $text(a)$, derivation sources $src(a)$ (revisions supplied to the generator that produced a), dependencies $dep(a)$ (revisions a requires at use time), motivating experiences $exp(a)$, and a capability contract (Section II-E). Editing produces a new revision; it never overwrites one.

Definition 4 (Exposure). Revision b is *directly exposed to* revision x, written $x \rightsquigarrow b$, iff (i) $x \in src(b)$ (synthesis exposure), (ii) $x \in dep(b)$ (use dependency), or (iii) there is an experience $e \in exp(b)$ produced by a run $r = run(e)$ whose loaded context $ctx(r)$ contained x (*experience-mediated exposure*, EME). Revision b is *exposed to* x if a path $x \rightsquigarrow \cdots \rightsquigarrow b$ exists.

In the running example, $m_7 \rightsquigarrow k_1$ by (i), $k_1 \rightsquigarrow p_{12}^1$ by (i) and (ii), and $m_7 \rightsquigarrow m_8$ by (iii), because run r_8 loaded m_7 and its experience e_8 motivated m_8. Exposure is an *over-approximation* of influence: k_1 is exposed to m_9 as much as to m_7, although m_9 contributes nothing faulty. It records what a generator could have used, never that it did.

D. Withdrawal and Change Impact

When a revision is found to be harmful, an operator must know everything that inherits from it and is still in use. In classical software, change impact analysis computes what a change may affect from dependency information [18], and program slicing does so for statements [19]. The harness analogue is a reachability question over exposure.

Definition 5 (Exposure closure and affected set). For a set of roots R, the closure $\mathrm{Cl}(R)$ is the least set containing R and closed under $\rightsquigarrow$. Given the active set Act and deployments

$Dpl : D \mapsto 2^{\mathcal{A}}$, the *affected set* of R is $\mathrm{Cl}(R) \cap (Act \cup \bigcup_d Dpl(d))$.

Definition 6 (Withdrawal). Withdrawing a revision r marks it permanently withdrawn and suspends its affected set from Act and from every registered deployment, while retaining all revisions and history. Withdrawing an experience e does the same for every revision motivated by e and makes every future revision motivated by e inadmissible.

For the example, $\mathrm{Cl}(\{m_7\}) = \{m_7, m_8, k_1, p_{12}^1, p_{12}^2\}$ plus any later revisions exposed to these; withdrawal removes them from both deployments. Restoring an earlier snapshot is not always a valid alternative: the snapshot taken right after k_1 was accepted still contains k_1, so an exact rollback to it would reactivate withdrawn lineage. Rollback also cannot recall external actions (for example, a message already sent) taken under the faulty state [6].

E. Contracts and Admission

A skill or profile often relies on capabilities that other artifacts supply; suspending one artifact can therefore break another that was not derived from it. Design by contract makes such assumptions explicit as pre- and postconditions [20]. We use a deliberately weak, decidable form.

Definition 7 (Capability contract). Each revision declares finite capability sets $req(a)$ and $prov(a)$. A set S is *contract-consistent* iff for every $a \in S$, $dep(a) \subseteq S$ and $req(a) \subseteq \bigcup_{d \in dep(a)} prov(d)$.

Contracts are syntactic: consistency says that declared requirements are matched, not that natural-language content is correct. In the example, p_{12}^1 depends on k_0 and k_1; suspending k_1 without suspending p_{12}^1 would leave an inconsistent deployment.

F. Model-Driven Engineering

Harness artifacts are heterogeneous files, but the questions above concern their *relationships*. Model-driven engineering (MDE) raises such relationships to first-class, typed models conforming to a metamodel, and treats changes as model transformations [21], [22]. Traceability links between model elements then support impact analysis and consistency checking [23]–[25].

Definition 8 (Harness model). A harness model is a state $\sigma = (\mathcal{E}, \mathcal{A}, Q, V, Q_{\mathrm{acc}}, W, K, \Theta, Act, Ver, Dpl, L, X, \Lambda)$: experiences, revisions, proposals, validation records, accepted proposals, withdrawn and quarantined revisions, experiences withdrawn as evidence, active revisions, immutable snapshots, deployment membership, loaded runs with their contexts, external effects, and an append-only event history. Evolution operators (Experience, Propose, Validate, Accept, Retire, Withdraw of a revision or an experience, Quarantine, Rollback, Deploy, Load, Effect) are guarded transitions $\sigma \xrightarrow{op} \sigma'$.

G. Model Checking and Runtime Verification

Guards are only useful if no sequence of operations can bypass them; reasoning about all sequences by hand is error-prone. Explicit-state model checking exhaustively explores the reachable states of a finite transition system and reports a counterexample trace when a property fails [26]; SPIN is a mature checker for models written in Promela [27].

Definition 9 (Safety property). A safety property $\mathbf{G}\,\varphi$ requires that predicate φ holds in every reachable state (or transition). A counterexample is a finite trace ending in a state violating φ.

A model, however, is not the implementation. Runtime verification checks the executions of the real system against a specification [28]; we use it to compare logged harness events with the model's semantics.

Definition 10 (Conformance). An event log conforms if replaying it through the reference semantics accepts every event and reproduces every observed state digest.

TABLE I
NOTATION

Symbol	Meaning
$\mathcal{A}$, a	artifact revisions (immutable), one revision
$\mathcal{E}$, e	experiences; $run(e)$ produced e
$src(a)$, $dep(a)$	derivation sources, dependencies of a
$exp(a)$	experiences that motivated a
$ctx(r)$	revisions loaded into the context of run r
$x \rightsquigarrow b$	b directly exposed to x (Definition 4)
Θ, F	experiences withdrawn as evidence; forbidden roots
$\mathrm{Cl}(R)$	exposure closure of roots R (Definition 5)
Act, $Dpl(d)$	active set; membership of deployment d
W, K	withdrawn and quarantined revisions
Ver	immutable snapshots (harness versions)
$req(a)$, $prov(a)$	required and provided capabilities
σ, Λ	harness state; append-only event history
P1–P10	safety properties (Table II)

III. The Heartness Approach

Heartness places a policy-independent controller between the self-evolving policy that *proposes* changes and the persistent state that later runs *load*.

A. Overview

Fig. 1 shows the components and the data that flow between them. The *self-evolving policy* (any LLM-driven reflection, curation or composition step) is untrusted: it only returns text. The *host capture boundary* (Section III-C) decides which revisions are placed in each generation and run context, and records them as exposure (①,②). Proposals enter the *harness model* (Section III-B) only through *guarded admission* (Section III-D), which validates the exact candidate content against the current snapshot (③,④). Accepted snapshots are deployed and loaded by later runs (⑤), whose experiences close the self-evolution loop. When an operator withdraws a faulty revision, the *closure and withdrawal engine* (Section III-E) computes the exposure closure, suspends it in every deployment and rejects incompatible rollbacks (⑥,⑦); closure-bounded revalidation can then re-admit screened members under an attestation (Section III-F, ⑩). Offline, a *finite projection* of the lifecycle is model checked by two independent encodings, and counterexamples are replayed on the implementation (Section III-G, ⑧). Online, a *conformance monitor* replays the event history and compares observed state digests (Section III-H, ⑨).

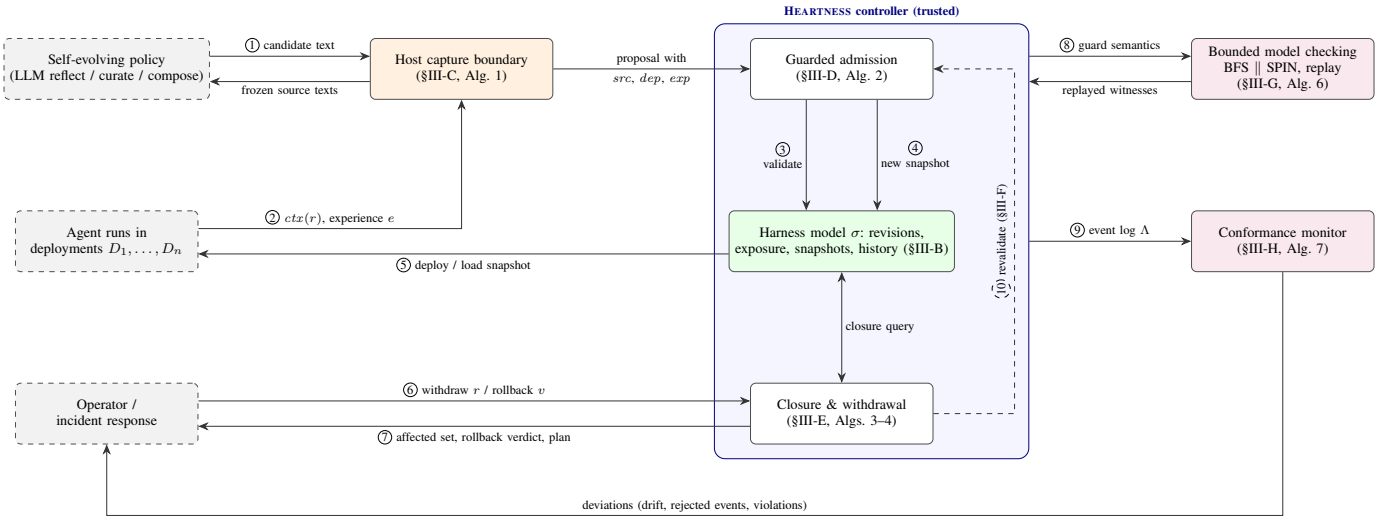


Fig. 1. Overview of HEARTNESS. Dashed boxes are untrusted or external; the shaded group is the trusted controller. Numbered arrows: ① the policy returns only candidate text, ② the host records each run's loaded context and resulting experience, ③–④ admission validates the exact candidate against the current snapshot and installs a new immutable snapshot, ⑤ deployments load snapshots, ⑥–⑦ withdrawal and rollback requests are answered with the affected set and an admissibility verdict, ⑧ a finite projection of the guards is model checked and witnesses are replayed on the implementation, ⑨ the event log is replayed by the conformance monitor, ⑩ revalidation re-admits screened members of the closure as fresh revisions under an attestation.

The design rests on three explicit assumptions that the evaluation probes rather than presumes: (A1) persistent state changes only through the controller's API (direct file edits are outside the model and are detected only if observed, Section III-H); (A2) the host, not the policy, records exposure, so exposure is complete for the inputs the host itself supplies but not for ambient inputs it does not mediate; (A3) admissions are serialized against snapshot stamps (optimistic concurrency), and deployments are connected when withdrawn.

B. Harness Metamodel and Lifecycle State

Motivation. The queries in Section II fail when relationships live only in file contents or in an informal log. The metamodel turns every relationship needed to answer them into a typed, immutable reference.

Design. The metamodel (Definition 8) has classes Experience, ArtifactRevision (one tagged class for the seven artifact kinds and the attestations of Section III-F), Proposal, ValidationRecord, HarnessVersion, ExternalEffect, and Event, plus the relations *motivatedBy*, *derivedFrom*$(kind)$, *dependsOn*, *replaces*, *loadedContext*, and deployment membership. Revisions are content-addressed (SHA-256 of a canonical serialization), so a snapshot fixes both identities and contents. The state also holds the sets W of withdrawn and K of quarantined revisions and the set Θ of experiences withdrawn as evidence. The *forbidden roots* are $F = W \cup K \cup \{b \mid exp(b) \cap \Theta \neq \emptyset\}$; P2 and P8 forbid their whole closure $\mathrm{Cl}(F)$. We call the approach model-driven in the models@run.time sense [29]: the metamodel is the runtime representation that every operator reads and writes, P1–P10 are invariants over its instances, the Promela model checked in Section III-G is generated from a projection of the operator semantics, and the conformance monitor replays the event log through the same operators; we do not use a metamodelling workbench. Lifecycle status is *derived*, not stored: a revision can be historically accepted and currently suspended. Every operator either applies atomically and appends a hash-chained event to Λ, or is rejected and leaves σ unchanged. Table II lists the ten safety properties the controller maintains; all have the form $\mathbf{G}\,\varphi$.

TABLE II
SAFETY PROPERTIES MAINTAINED BY HEARTNESS AND THE EVIDENCE USED FOR EACH. "PROJ." = CHECKED ON THE FINITE PROJECTION BY BOTH BFS AND SPIN.

ID	Predicate	Evidence
P1	every revision reaches an experience or an authorized seed via *src*	guard, audit, tests
P2	$Act \cap \mathrm{Cl}(F) = \emptyset$	guard, audit, Proj.
P3	active and deployed sets are contract-consistent	guard, audit, Proj.
P4	accepted/loaded content equals validated/snapshot content	digest guard, Proj.
P5	rollback restores the exact recorded set; effects are never removed	audit, tests
P6	derivation endpoints exist; derivation graph is acyclic	guard, audit
P7	history and revision identities are append-only	transition audit
P8	no registered deployment contains $\mathrm{Cl}(F)$	suspension, Proj.
P9	an accepted proposal's base is the current snapshot	stamp guard, Proj.
P10	event chain and observed digests agree with replay	monitor

Running example. After task 12 the model contains $m_0, \ldots, m_{11}$, k_0, k_1, $p_0^1, p_{12}^1, p_0^2, p_{12}^2$; k_1 has $src = \{m_6, m_7, m_9\}$ (abbreviated), p_{12}^1 has $src = \{p_0^1, k_0, k_1\}$, $dep = \{k_0, k_1\}$ and *replaces* p_0^1; $Dpl(D_1) = \{p_{12}^1, k_0, k_1\}$; run r_8 has $ctx(r_8) = \{p_0^1, m_7, \ldots\}$ and produced e_8, which motivates m_8.

Assumptions and limits. The metamodel captures declared structure only; it does not interpret natural-language content, and contracts are syntactic (Definition 7).

Algorithm 1 Host-side exposure capture

1: **procedure** RUNTASK(d, task t)
2: $C \leftarrow$ ASSEMBLECONTEXT(d, t) ▷ profile, skills, memories
3: APPLY(LOAD(r, C))
4: $o \leftarrow$ execute agent on t with C; APPLY(EXPERIENCE(e, t, r, o))
5: **return** e
6: **procedure** GENERATE(policy π, sources S, experiences E, kind κ)
7: $T \leftarrow [text(s) \mid s \in S]$ ▷ frozen before π runs
8: $y \leftarrow \pi(T, E)$ ▷ untrusted text only
9: $a \leftarrow \langle fresh, \kappa, y, src{=}S, exp{=}E \rangle$
10: **return** ADMIT(a) ▷ Algorithm 2

Algorithm 2 Guarded admission (ADMIT = PROPOSE, VALIDATE, ACCEPT)

procedure PROPOSE(candidate a)
  require $src(a) \cup dep(a) \subseteq \mathcal{A}$, $exp(a) \subseteq \mathcal{E}$, $src(a) \cup exp(a) \neq \emptyset$ or seed ▷ P1, P6
  return $q \leftarrow \langle a, base{=}|Ver|{-}1 \rangle$ ▷ stamp current snapshot
procedure VALIDATE(q)
  return $v \leftarrow \langle q, hash(q.a), check(q.a) \rangle$ ▷ pluggable check
procedure ACCEPT(q, v) ▷ possibly after other admissions
  require $v.passed \wedge v.hash = hash(q.a)$ ▷ P4
  require $q.base = |Ver| - 1$ ▷ P9: no admission since Propose
  $S \leftarrow (Act \cup \{q.a\}) \setminus replaces(q.a)$
  require $S \cap \mathrm{Cl}(F) = \emptyset$ ▷ P2; Cl over $\mathcal{A} \cup \{q.a\}$
  require CONSISTENT(S) ▷ P3
  $Act \leftarrow S$; append snapshot $\langle S, \{(x, hash(x))\}_{x \in S} \rangle$

C. Host-Side Exposure Capture

Motivation. Provenance cannot be recovered after the fact from LLM output (Section II-C); it must be recorded where inputs are assembled. Recording synthesis inputs alone is also insufficient: the running example shows that a faulty lesson can re-enter memory through the agent's behaviour ($m_7 \rightsquigarrow m_8$).

Design. The host performs two recordings (Algorithm 1). Before every agent run it records the exact revisions placed in the context (LOAD, line 3); the controller rejects a context containing revisions that are neither active nor deployed. Before every generation step it freezes the selected source revisions and their texts, invokes the policy, and attaches the frozen identities to the proposal itself (lines 7–10). The policy never names its own sources. EME edges are then *derived*: an artifact motivated by experience e is exposed to every revision in $ctx(run(e))$.

Running example. For task 9 (run r_8), ASSEMBLECONTEXT retrieves p_0^1, the most relevant skill k_0 and the three memories most similar to the task, including m_7; LOAD$(r_8, \{p_0^1, k_0, m_5, m_6, m_7\})$ is recorded before the run produces its experience (LOAD is rejected for a run that already has one). The reflection that produces m_8 receives e_8 and two retrieved lessons, so m_8 gets $src = \{m_5, m_6\}$ and $exp = \{e_8\}$; the EME edge $m_7 \rightsquigarrow m_8$ follows from $ctx(r_8)$ without any action by the policy.

Assumptions, limits, cost. Capture is complete only for inputs the host mediates (A2): a tool result that quotes an artifact, or the model's parametric knowledge, are not captured. Capture costs one set copy per run and per generation, linear in context size.

D. Guarded Admission

Motivation. A self-evolving loop proposes changes continuously; without a gate, a candidate can be modified after validation, validated against a stale snapshot, or admitted while it inherits from withdrawn state.

Design. Admission (Algorithm 2) has three operators. PROPOSE checks freshness, typed kinds, known sources, dependencies and experiences (P1, P6) and stamps the proposal with the current snapshot as its base. VALIDATE binds a Boolean outcome to the candidate's content digest and base; validation itself is pluggable (tests, reviewers, LLM judges). ACCEPT requires a passed validation of the *same* digest (P4), a base equal to the current snapshot (P9), and an admissible candidate active set: no member in the closure of withdrawn or quarantined revisions (P2) and contract consistency (P3). Success installs a new immutable snapshot; failure rolls back atomically.

Running example. The composition step proposes p_{12}^1 with base v, replacing p_0^1. If another admission (say k_1) lands in between, the base is stale and ACCEPT rejects p_{12}^1 (P9); the host re-proposes against the new snapshot. Once m_7 is withdrawn, any later proposal that merges m_8 is rejected by line 10: the closure is computed over the stored revisions *plus the candidate*, and the candidate is a child of $m_8 \in \mathrm{Cl}(\{m_7\})$, so it lies in the closure itself. The same holds for a candidate motivated by an experience whose run loaded m_7 before the withdrawal.

Assumptions and cost. Line 10 recomputes a closure, $O(|\mathcal{A}| + |\rightsquigarrow|)$ per acceptance; Section IV-E measures this cost. Validation quality is outside the controller: a passing but wrong validator admits wrong content, which is exactly the situation withdrawal must then repair.

E. Exposure Closure, Withdrawal, and Rollback

Motivation. Once a revision is known to be faulty, every revision that inherits from it and every deployment that loads such a revision must be found and contained; restoring an old snapshot must not silently reactivate withdrawn lineage.

Design. Algorithm 3 computes $\mathrm{Cl}(R)$ by breadth-first search over three successor relations: derivation children, dependency consumers, and EME children, the last obtained by joining each revision's experiences with the contexts of the runs that produced them. Algorithm 4 withdraws a root: it records the root in W, quarantines every other revision of the closure in K, removes the closure from Act and from every deployment (P2, P8), and appends a snapshot; nothing is deleted. The part of the

Algorithm 3 Exposure closure $\mathrm{Cl}(R)$

$child[x] \leftarrow \{b \mid x \in src(b)\}$; $cons[x] \leftarrow \{b \mid x \in dep(b)\}$
$eme[x] \leftarrow \{b \mid \exists e \in exp(b) : x \in ctx(run(e))\}$
$S \leftarrow R$; $Q \leftarrow R$
while $Q \neq \emptyset$ **do**
  $x \leftarrow \textsc{Pop}(Q)$
  for $b \in child[x] \cup cons[x] \cup eme[x] \setminus S$ **do**
    $S \leftarrow S \cup \{b\}$; $\textsc{Push}(Q, b)$
return S

Algorithm 4 Withdrawal and rollback admissibility

procedure $\textsc{Withdraw}(r)$ ▷ r a revision or an experience
  if $r \in \mathcal{E}$ **then** $\Theta \leftarrow \Theta \cup \{r\}$; $R \leftarrow \{b \mid r \in exp(b)\}$
  else $R \leftarrow \{r\}$
  $Z \leftarrow \mathrm{Cl}(R)$; $W \leftarrow W \cup R$; $K \leftarrow K \cup (Z \setminus R)$
  $Act \leftarrow Act \setminus Z$
  for all deployments d **do** $Dpl(d) \leftarrow Dpl(d) \setminus Z$
  append snapshot; **return** Z and external effects X (not reversed)
procedure $\textsc{Rollback}(v)$
  $S \leftarrow Ver[v].active$
  if $S \cap \mathrm{Cl}(F) \neq \emptyset \vee \neg\textsc{Consistent}(S)$ **then**
    reject; **return** $\textsc{LastGood}$, X
  $Act \leftarrow S$; append snapshot "rollback:v"

closure that was in use, $Z \cap (Act \cup \bigcup_d Dpl(d))$, is the affected set of Definition 5 that is reported to the operator (Fig. 1, ⑦). WITHDRAW also accepts an *experience* e: it marks e as tainted evidence, withdraws every revision whose motivation $exp(\cdot)$ contains e together with its closure, and makes every *future* revision motivated by e a forbidden root, so that a policy that re-reads its experience pool cannot re-learn the same lesson from the same corrupted feedback (P2). Re-learning from *new* evidence, such as a second corrupted message, is outside what any lineage-based guard can prevent. ROLLBACK to snapshot v is admissible only if the target set passes the same checks as admission; otherwise it is rejected and the operator receives the most recent admissible snapshot (LASTGOOD) and the list of external effects that rollback cannot reverse.

Running example. $\textsc{Withdraw}(m_7)$ visits $m_7 \to k_1$ (child), $m_7 \to m_8$ (EME), $k_1 \to p_{12}^1, p_{12}^2$ (child and consumer), and any revision later exposed to these; both deployments lose k_1 and their rewritten profiles and keep k_0. Withdrawal never re-activates the replaced base profiles p_0^1, p_0^2 by itself; the operator restores them explicitly, by redeploying them or by rolling back to an admissible snapshot. A rollback to the snapshot installed right after k_1 is rejected because that snapshot contains $k_1 \in K$; LASTGOOD returns the most recent snapshot disjoint from the closure. Rollback changes Act only; deployments change through DEPLOY, so a rollback reaches agents only when it is redeployed. The phrase "any revision later exposed to these" dominates the cost in practice: because every run after task 12 loads its deployment's rewritten profile, every memory written after that point is EME-exposed to p_{12}^1 or p_{12}^2 and joins the closure; only memories written between the fault and the rewrite by runs that never loaded m_7 or its descendants are spared. Section IV-C1 measures exactly this effect.

Design rationale and limits. Closure deliberately over-approximates: k_1 is suspended although only one of its three sources was faulty, and it would be suspended even if its text had dropped the faulty advice. Revisions that are not exposed to m_7, such as m_9 itself, stay active. Proposition 1 states the bound this over-approximation obeys. Quarantine is permanent; suspended state returns to use only through closure-bounded revalidation (Section III-F) or fresh lineage. The BFS is linear in the number of revisions and exposure edges.

Proposition 1 (Closure is bounded by a time cut): Let $t(b)$ be the position in Λ of the event that accepted revision b ($t(b) = 0$ for seeds). For every root set R, $\mathrm{Cl}(R) \subseteq \{b \in \mathcal{A} \mid t(b) \geq \min_{r \in R} t(r)\}$.

Proof: Every edge $x \rightsquigarrow b$ has $t(x) < t(b)$; the EME case relies on the LOAD precondition that a run's context is recorded before the run produces an experience. For synthesis and dependency edges, PROPOSE requires $x \in \mathcal{A}$, so x was accepted before b was proposed. For an EME edge, $x \in ctx(run(e))$ with $e \in exp(b)$: LOAD admits only active or deployed revisions into a context, the run precedes its experience, and PROPOSE requires $e \in \mathcal{E}$. Hence every path from a root to b is increasing in t. ■

Consequently, withdrawal never suspends more than restoring the last snapshot before the earliest root (a *time cut*), and it suspends exactly the same set as that cut when every revision accepted after the root is exposed to it. The time-cut baseline of the evaluation cuts before the corrupted *experience*, which precedes the root's acceptance, so it suspends at least as much and the bound holds for it as well. How far closure stays below the time cut is therefore determined by the exposure structure of the self-evolving policy, which the evaluation measures (Section IV-C1). Closure differs from the time cut in two respects: it keeps later work that was not exposed, and it is enforced on future admissions, rollbacks and deployments rather than applied once. Both compute from the same input, since the root's acceptance time is in Λ; reaching every deployment is a requirement on any remedy in a multi-deployment harness, which HEARTNESS satisfies by construction (P8) and a rollback satisfies only if it is redeployed.

F. Closure-Bounded Revalidation

Motivation. Proposition 1 bounds the over-approximation but does not remove it: when a policy exposes most later revisions to the root, withdrawal suspends most learned state. Detectors that judge content are more precise but may miss contaminated revisions, and applied to the whole state they can also flag revisions that have nothing to do with the root.

Design. Revalidation uses the closure as the *candidate set* and a screen only to give members back (Algorithm 5). A pluggable screen (an LLM judge or a similarity threshold, relative to the text of the withdrawn roots) decides for each quarantined member whether it is free of the faulty content.

Algorithm 5 Closure-bounded revalidation

Input: roots R, screen ϕ; Act_{b}, Dpl_{b}: active set and deployments before withdrawal

$C \leftarrow \{x \in \mathrm{Cl}(R) \setminus R \mid x \in Act_{\mathrm{b}} \cup \bigcup_d Dpl_{\mathrm{b}}(d),\ \phi(text(x), text(R)) = clean\}$
$\nu \leftarrow \emptyset$ ▷ ν: original ↦ copy
if some $x \in C$ has all dependencies active or in C **then** ADMIT(α) with $\alpha = \langle R, \phi, C\rangle$ as authorized seed
for all $x \in C$ in history order **do**
  $D \leftarrow \{\nu(y)$ if $y \in \mathrm{dom}\,\nu$ else $y \mid y \in dep(x)\}$
  if $D \subseteq Act$ **then** ▷ else rejected (P3)
    $S_x \leftarrow \{\alpha\} \cup$ kept sources $\cup$ clean frontier of dropped sources and contexts
    $x' \leftarrow$ copy of x with $src(x')=S_x$, $dep(x')=D$, clean experiences
    ADMIT(x'); $\nu(x) \leftarrow x'$
for all d **do** DEPLOY(d, $Dpl(d) \cup \{\nu(x) \mid x \in Dpl_{\mathrm{b}}(d) \cap \mathrm{dom}\,\nu\}$)

Candidates are the members of the affected set (Definition 5) before the withdrawal, other than the roots, that the screen clears; because the affected set contains only revisions in use (active or deployed), a revision that a later version had already replaced does not return beside its successor. If any candidate is admissible, revalidation first admits one *attestation* revision $\alpha = \langle R, \phi, C\rangle$, a host-authorized seed. Each cleared member x is then re-admitted, in history order, as a fresh revision x' with the same text and contract. Its lineage is x's lineage with the withdrawn part replaced by α: its sources are α, the sources of x outside $\mathrm{Cl}(F)$ (active or not), and the re-admitted copies of those that were cleared; it keeps the experiences of x whose run context avoided $\mathrm{Cl}(F)$; every source or experience that cannot be kept is replaced by its *clean frontier*: the nearest ancestor revisions outside $\mathrm{Cl}(F)$, active or not, and the clean experiences that motivated dropped intermediates; and its dependencies are $dep(x)$ with every quarantined dependency replaced by its re-admitted copy. A later withdrawal of any clean ancestor of x therefore still reaches x'. A member with a dependency that is neither active nor re-admitted is rejected, as admission would reject it (P3): a profile returns only if the skills it depends on do. Admitted copies are redeployed to the deployments that held the original before the withdrawal (Dpl_{b}). The original x stays quarantined and the history keeps both. Because every source, experience context and dependency of x' lies outside $\mathrm{Cl}(F)$, so does x', and P2 holds; because it derives from α, withdrawing the attestation (for example, when the screen is found faulty) suspends every re-admission through the ordinary closure. Revalidation is thus a composition of existing operators and adds no semantics; the implementation (`heartness.revalidation`) executes it with PROPOSE, VALIDATE, ACCEPT and DEPLOY.

Running example. After WITHDRAW(m_7), the judge reads k_1, p^1_{12}, p^2_{12} and m_8 against m_7. If k_1's merged text had dropped the print instruction, the judge would clear it and k'_1 would be re-admitted and redeployed; m_8 and the profiles, which repeat the advice, stay quarantined. Had the judge cleared p^1_{12} but not k_1, p^1_{12} would be rejected, because it depends on k_1.

Limits. Revalidation is only as good as its screen; a missed contaminated member returns to use, which the evaluation measures as residual contamination. It never re-admits revisions outside the closure and never needs to inspect them, so its cost is bounded by the affected set. In the live study we evaluate revalidation on the recorded final state in a separate process: the same planning function that the implementation executes (`heartness.revalidation.plan`) decides which cleared members are re-admitted, and a test checks on small fixtures that the plan equals the engine's execution. The plan checks only dependency availability; in the engine every copy also passes the ordinary admission guards, which could reject it, but in the live records they cannot bind: a copy's sources, experience contexts and dependencies are by construction outside $\mathrm{Cl}(F)$, its content hash is the one validated, and contracts are those of the original.

Algorithm 6 Bounded checking and witness replay

Input: scope $(n, topology, depth)$, mutant μ (or none)

$(safe_B, w) \leftarrow$ BFS($init(n)$, $step_\mu$, $depth$)
$P \leftarrow$ PROMELA($n, topology, \mu$); $safe_S \leftarrow$ SPIN(P)
record $agree \leftarrow (safe_B = safe_S)$
if $\neg safe_B$ **then**
  record REPLAYPROJECTION(w), BFS($\ldots, |w|-1$) is safe
  record violations of REPLAYENGINE(w, disabled=μ)

G. Bounded Model Checking and Implementation Correspondence

Motivation. The guards interact: deployment replacement, repeated withdrawal and rollback all change snapshots and base stamps. Testing individual operations does not show that no *sequence* reaches a state violating P2–P9.

Design. We define a finite projection of the lifecycle with $n \leq 8$ revision identities, a static derivation/dependency topology (chain, star, or independent), Boolean validation, and the operators Propose, Validate, Accept, Withdraw, Deploy and Rollback. The projection is encoded twice, independently: as an explicit breadth-first search in Python, which yields shortest counterexamples, and as a Promela model checked by SPIN 6.5.2 [27], [30]. Algorithm 6 runs both on the reference guards and on seeded *guard mutants* (each removing one guard), compares verdicts, replays each witness on the projection and on the real engine with the same guard disabled, and checks minimality by re-running at depth $\ell-1$. A separate experiment maps every successor of small reference states between the engine and the projection and compares enabled operations and mapped state fields.

Running example. With three identities in a chain ($m_7 \to k_1 \to p_{12}$) and the suspension guard for P2 removed, BFS returns the 7-operation witness Propose, Validate, Accept m_7; Propose, Validate, Accept k_1; Withdraw m_7. Withdrawal then removes only m_7, so k_1 stays active although its source is withdrawn (P2) and although it depends on a revision that is

no longer active (P3). SPIN agrees, no shorter witness exists, and replaying the sequence on the engine with the same guard removed produces audit violations P2 and P3.

Limits. Agreement between two encodings of the same projection is cross-validation, not a refinement proof; the projection omits text, reduces contracts to the presence of dependencies, and omits effects, EME edges, multiple deployment identities, experience withdrawal and revalidation; the mutation campaign also does not cover the last two, which are covered by unit tests. Proposition 2 covers what the projection omits, for P2 and P8.

Proposition 2 (Suspension invariant): With the forbidden roots F of Section III-B, let $I(\sigma)$ be $Act \cap \mathrm{Cl}(F) = \emptyset \wedge \forall d : Dpl(d) \cap \mathrm{Cl}(F) = \emptyset$ (P2 and P8), with Cl over all exposure edges including EME. If $I(\sigma)$ holds and $\sigma \xrightarrow{op} \sigma'$ is any successful operator of the engine, then $I(\sigma')$ holds. The proposition concerns the *recorded* closure: it says that nothing recorded as exposed to withdrawn lineage is in use, not that all contaminated content is.

Proof sketch: By cases. EXPERIENCE, PROPOSE, VALIDATE, LOAD and EFFECT leave Act, Dpl, W, K, Θ unchanged and add no edge between stored revisions: stored revisions are immutable and cannot cite an experience created after them, and LOAD is rejected for a run that has already produced an experience, so a late context record cannot add EME edges into accepted revisions. ACCEPT adds one revision a and edges only *into* a, so $\mathrm{Cl}(F)$ grows at most by a; the guard rejects the operation if the candidate set meets the closure computed with a included. RETIRE only shrinks Act and Dpl. WITHDRAW(r) and QUARANTINE(r) remove $\mathrm{Cl}(\{r\})$ from Act and every $Dpl(d)$, and $\mathrm{Cl}(F \cup \{r\}) = \mathrm{Cl}(F) \cup \mathrm{Cl}(\{r\})$; withdrawing an experience e applies this to every revision motivated by e, and any later revision motivated by e is in F and is rejected by the ACCEPT guard. ROLLBACK and DEPLOY install a set only after the same admissibility check. ∎

The proposition is a hand proof over the engine's operator semantics, not a mechanized one; the bounded checks and the mutation tests (Section IV-A) give independent evidence that the implementation follows it. Scopes are small; the state space grows exponentially with n (Section IV-E).

H. Runtime Conformance Monitor

Motivation. Model checking speaks about the model; the deployed harness can still drift, for example when a file is edited outside the API.

Design. Algorithm 7 replays a logged event sequence through a fresh reference engine, compares each recomputed state digest with the observed one, and evaluates the audit predicates P1–P10 on every transition. It fails closed: after a rejected event there is no synchronized reference, so monitoring stops and reports.

Running example. If someone edits the file of k_1 in D_2 after withdrawal, the next logged LOAD carries a digest that differs from replay and is reported as drift. If the agent sent an e-mail under p_{12}^2 and the host never logged it, nothing is reported: unlogged actions are invisible to any log-based monitor.

Algorithm 7 Conformance monitor

Input: logged events $\langle (op_i, args_i, h_i) \rangle_i$

$\sigma \leftarrow \sigma_0$; $D \leftarrow [\,]$
for $i = 1, \ldots$ **do**
  $\sigma' \leftarrow$ APPLY$(op_i, args_i)$ **or** append (*rejected*, i) to D and **stop**
  if $h_i \neq \bot \wedge h_i \neq hash(\sigma')$ **then** append (*drift*, i)
  append violations of P1–P10 on (σ, σ'); $\sigma \leftarrow \sigma'$
return D

IV. EVALUATION

The evaluation answers the three research questions stated in Section I. RQ1 concerns containment effectiveness, RQ2 the assurance provided by the guards and the bounded checking, and RQ3 cost and the limits of runtime conformance.

A. Implementation

HEARTNESS is implemented in Python 3.12 (1,209 lines in `src/heartness`): the metamodel as frozen dataclasses with canonical SHA-256 digests (`model.py`), transactional operators with atomic rollback (`engine.py`, Algorithms 2 and 4), the closure including EME edges (`graph.py`, Algorithm 3), the host capture adapter (`capture.py`, Algorithm 1), the audit predicates P1–P10 (`properties.py`), closure-bounded revalidation (`revalidation.py`, Algorithm 5), the conformance monitor (`monitor.py`, Algorithm 7), and the finite projection with its BFS and Promela generator (`checker.py`, Algorithm 6). SPIN 6.5.2 is built from a pinned commit. A JSON command-line interface exposes withdrawal plans and rollback verdicts over saved event logs. Each guard can be disabled individually, which provides the mutants and ablations used below. The implementation has 60 tests with 98.4% statement and 94.6% branch coverage, and strict `mypy`; 14/14 hand-seeded source mutants of the guards are killed by the test suite.

The live study (Section IV-B) adds a self-evolving coding harness with its analysis and batch drivers (1,800 lines in `experiments/live`) whose every persistent write goes through the controller. It serves open-weight models through a private Ollama 0.32.14 server started inside each batch job, with temperature 0, fixed per-request seeds and a fixed 8,192-token context; every model response is cached by a digest of model, prompt and options, so each reported run can be replayed without a model server. The live study ran on the Fir cluster of the Digital Research Alliance of Canada, one job per cell on a 20 GB slice of an NVIDIA H100 80GB HBM3; the synthetic experiments, the model checking and the local pilot ran on one NVIDIA DGX Spark–class machine (GB10, 20 CPU cores, 121 GB unified memory). The HPC pipeline (job graph, smoke test, audit and collection) is part of the replication package.

B. Experimental Setup

1) Subjects: Table III lists the three subjects and which RQs use them.

(S1) Live self-evolving harness. We needed a setting where faulty state arises and spreads through real LLM generation, and where the damage can be measured independently of our own provenance records. We therefore implemented a self-evolving coding harness and evolved it on HumanEval+ [31], [32], a widely used code-generation benchmark whose extended tests give an objective pass/fail signal. Of its 164 tasks, 29 fixed held-out tasks measure behaviour and the remaining tasks supply the evolution stream (24 tasks per run, shuffled per seed).[1] A run serves two deployments D_1 and D_2 and alternates tasks between them. Every task is solved from a context assembled by the host, its solution is tested, and the outcome and test feedback are recorded as an experience. How experiences become lasting state is the *harness policy*, and we use two:

- *ExpeL-style insights* (adapted from ExpeL [2] to single attempts; not an exact re-implementation). Every four tasks the model is shown the batch's attempts with their test feedback and the current insight list, and answers with ADD, EDIT, AGREE and REMOVE operations. Insights carry importance scores (ADD $= 2$, EDIT and AGREE $+1$, REMOVE -1; an insight at importance ≤ 0 is retired). D_1 loads all insights and D_2 the five most important, and every task context is the deployment's insight list. A new or edited insight records every insight shown to the extraction step as a source and the batch's experiences as its motivation. Importance scores are ranking metadata kept by the harness, not governed revisions; changing them changes which insights D_2 loads but not what any insight says.
- *Curation* (the running example, Section II-B). After every task the model writes a one-sentence memory from the experience, with the two most similar memories as sources; each context holds the deployment's profile, its two most relevant skills and the three most relevant memories; every six tasks the model merges recent memories into a skill; after tasks 12 and 24 each deployment's profile is rewritten from the current skills. A faulty lesson can therefore reach a profile in two synthesis hops, and a run that loads a faulty artifact can produce a new memory through its experience.

The two policies differ in exactly the structure that matters for containment: insight editing keeps one flat, heavily cross-sourced list, whereas curation builds a layered hierarchy (memory, skill, profile) with use dependencies.

In each *faulty* run exactly one test-feedback message, at a seed-dependent position (task 2, 8, 9 for our seeds), is replaced by misleading grader feedback of one of three types: F1 *print instead of return*, F2 *stringify return values*, F3 *use floor division*. All three are plausible misreadings of real test failures, and all three are wrong for HumanEval+. A paired *clean* run with the same seed receives no corrupted feedback. The *roots* of a faulty run are the revisions the controller recorded as motivated by the corrupted experience; they are known by construction and given to every remedy. Under curation this is the single memory written from that experience; under the ExpeL-style policy it is every insight the extraction step wrote from the batch containing it (2.1 on average, at most 4), of which typically at most one carries the faulty advice. Identifying the root is thus given to all remedies alike; we do not evaluate how an operator finds it. We used 5 open-weight generators from five organizations that are common in code-generation studies: `qwen2.5-coder:7b`, `llama3.1:8b`, `deepseek-coder-v2:16b`, `gemma2:9b` and `granite3.3:8b`. For each policy, generator and reporting seed (5 seeds) we ran the three faulty runs and the clean run, 200 runs in total. Thresholds were tuned beforehand on a separate local pilot (6 development runs, Section IV-B2) that is not part of the reported data.

(S2) Controlled synthetic benchmark. Live runs give realistic content but no control over graph structure, and their ground truth is only as good as the content oracle. The synthetic benchmark (EvoTraceBench 1.1, `benchmark/`) generates provenance DAGs with seven artifact kinds, nine structural motif families (e.g., multi-source synthesis, adversarial lexical decoupling, dependency-only consumers, deployment-only membership), sizes 16–256 (up to 2,048 for scaling), controlled capture rates and lexical decoupling, with exact construction-time ground truth computed independently of HEARTNESS's graph code. Hand-specified cases supply additional controls that do not depend on the generator.

(S3) Public skill-library replay. To check that the model accepts a real third-party evolution artifact, we replay the 12 revisions of Voyager's published skill library [4], [33] as 36 controller events.

Compute platform. The live study needs GPU inference for several open-weight models, so we prepared four Alliance Canada GPU clusters (Nibi, Narval, Fir, Rorqual) behind one submission and monitoring interface and ran an end-to-end smoke test (8 evolution tasks, one seed) on each. The smoke runs on Fir, Rorqual and Narval completed the full pipeline (setup, generation, oracle labelling, collection); the Nibi smoke run never started because its GPU nodes were unavailable. Smoke results were used only to find and fix deployment faults and are not reported as findings. We ran the reported study on one cluster (Fir) for four reasons. (i) The platform is not an experimental factor: every compared condition is applied to the same recorded run, so a platform effect would shift all conditions alike. (ii) Splitting the study would have introduced a confound, because the clusters differ in GPU model (H100, A100) and GPU kernels change low-order results even at temperature 0. (iii) The software stack is pinned and recorded in every run record (inference-server version, model digests, checksummed benchmark files, code hash, cluster and GPU), so the setup can be repeated elsewhere. (iv) Statistical power depends on faults, seeds and generators, not on the number of sites. The remaining limitation is stated in Section IV-F.

2) Baselines: The baselines represent what a team can do today without HEARTNESS. All receive the same roots and act on the same final state. On S1 we name them R1–R4 and T, A; on S2 the corresponding structural baselines are B0–B3.

[1] HumanEval/32 is excluded because its canonical solution fails its own HumanEval+ tests in our sandbox. All 163 remaining canonical solutions were re-validated in the batch environment before every campaign.

TABLE III
SUBJECTS. S1 IS GENERATED BY REAL LLM SELF-EVOLUTION; S2 HAS EXACT STRUCTURAL GROUND TRUTH; S3 IS A PUBLIC EVOLUTION ARTIFACT.

Subject	Size	RQs
S1 live harness	200 runs (5 generators × 2 policies × 5 seeds × (3 faults + clean)); 29 held-out tasks × 11 evaluated conditions per faulty run (15 conditions in total); 42,811 LLM calls	RQ1, RQ3
S2 synthetic	1,905 raw records; 810 traceability scenarios	RQ1, RQ2, RQ3
S3 Voyager	12 skill revisions, 36 events	RQ3

- *R1 delete root*: remove the faulty memory, as a file-level revert or `git rm` would. On S2 the corresponding baseline (B0) recovers lineage from within-file history only.
- *R2 lexical recovery*: suspend artifacts whose TF-IDF cosine to the root reaches τ_t, the classic IR traceability-recovery approach [9]. On S2 (B1), chronological token Jaccard with a fixed threshold of 0.2.
- *R3 embedding recovery*: suspend artifacts whose `nomic-embed-text` embedding cosine to the root reaches τ_e.
- *R4 LLM judge*: ask the generator model, for every live artifact, whether it repeats or relies on the root's (wrong) advice, a semantic content screen. The judge is shown the concatenated text of all roots of the run; under the ExpeL-style policy most roots are clean co-batched insights, which may lower its precision there.
- *T rollback before the fault*: restore the last snapshot before the corrupted experience, which suspends every revision created since. It needs the time of the fault, which the history provides for any known root, and by Proposition 1 it is the upper bound of the closure.
- *A reset learned state*: suspend every learned (non-seed) revision, i.e., return to the hand-written harness.
- *Capture-only provenance* (S2, B2): the recorded edges and the same closure, without admission, suspension or rollback semantics, representing PROV-style capture [10], [11].
- *Direct-edge linter* (S2, B3): flags active artifacts with a withdrawn direct source or dependency, without transitive propagation.

τ_t and τ_e were tuned before any reported run, on a local pilot with `qwen2.5:7b`, the curation policy and two development seeds per fault type (6 runs), choosing the value in $\{0.05, 0.10, \ldots, 0.95\}$ that maximized mean F1 against the oracle ($\tau_t = 0.05$, the lowest grid value, so the pilot optimum lies on the grid boundary; development F1 0.575; $\tau_e = 0.55$, development F1 0.625). They are then frozen for all generators and both policies. Because thresholds transferred from a pilot may disadvantage the similarity baselines, we also report an *oracle-tuned upper bound*: the single threshold that maximizes mean F1 on the reported runs themselves. This bound is never used for any other result. We also report the full F1-versus-threshold curves. Note that only two of the 6 pilot runs contained propagated contamination, so the frozen thresholds rest on little data.

Variants of HEARTNESS. H is the full approach (withdraw all roots). Ablations remove EME edges ($\mathrm{H}_{\neg\mathrm{EME}}$), dependency edges ($\mathrm{H}_{\neg\mathrm{dep}}$), or deployment propagation ($\mathrm{H}_{\mathrm{local}}$, guard P8 disabled). $\mathrm{H}_{\mathrm{faulty}}$ withdraws only the roots that the oracle labels contaminated, i.e., an operator who identified exactly the faulty revision; it is oracle-informed and serves as a sensitivity bound. Closure-bounded revalidation (Section III-F) uses the R4 judge ($\mathrm{HS}_{\mathrm{judge}}$) or the frozen R3 threshold ($\mathrm{HS}_{\mathrm{emb}}$) as screen. The revalidation conditions and T, A and $\mathrm{H}_{\mathrm{faulty}}$ were added after the local pilot and before the reported campaign was analysed; $\mathrm{H}_{\text{EME-fail}}$ was added after the first analysis, and revalidation was re-evaluated once with the dependency rule (Section IV-F); T, A and $\mathrm{H}_{\mathrm{faulty}}$ are computed from the recorded final state and have no behavioural measurement.

3) Ground truth and metrics: Content oracle (S1). An artifact is *contaminated* if it instructs or endorses the planted faulty practice. Two judge models from families different from every generator (`mistral-small:24b`, `phi4:14b`) label every live artifact with schema-constrained YES/NO answers; we use unanimous labels and report disagreements as *ambiguous*, treating them as clean in the main analysis and as contaminated in a sensitivity analysis. Judge agreement is reported as Cohen's κ. The oracle is independent of HEARTNESS's records: it never sees exposure edges.

Metrics. For a remedy that suspends set S, with C the contaminated non-root live artifacts: precision $|S \cap C|/|S \setminus A|$ (excluding ambiguous A), recall $|S \cap C|/|C|$, F1 (all three only over runs with $C \neq \emptyset$), *residual contamination* $|C \setminus S|$ (contaminated artifacts still loadable), and *collateral* (clean artifacts suspended). Behaviour is measured as pass@1 on the held-out tasks after remediation, and as the *fault-signature rate*, the fraction of solutions that statically exhibit the planted practice (for example a function that prints and returns nothing). On S2 we report F1 of origin, descendant and impact queries against construction truth. For RQ2 we report verdict agreement, witness length, replay outcome and state counts; for RQ3, wall-clock time and peak memory.

4) Statistics: Each faulty run is one paired observation across remedies. We compute 95% bootstrap confidence intervals (2,000 resamples, fixed seed) for every mean and ship them with the per-run values in the replication package; the paper reports means and paired counts. We compare H against each alternative with the two-sided Wilcoxon signed-rank test [34], and adjust p-values with Holm's method [35] within each metric over the family of seven comparators whose difference from H is an empirical question (no remedy, R1–R4, $\mathrm{HS}_{\mathrm{judge}}$, $\mathrm{HS}_{\mathrm{emb}}$); ablations and post-hoc conditions are reported descriptively because several are identical to H by construction in one policy. As paired effect sizes we report the number of runs in which H is better, worse or tied, and as a distributional effect size the Vargha–Delaney $\hat{A}_{12}$ [36], following Arcuri and Briand [37]. The effective sample size of every contamination metric is the number of runs with propagated contamination, which we state with each result. Bootstrap intervals and per-run values are in the replication

TABLE IV
FAULT ADOPTION AND PROPAGATION PER GENERATOR, EXPEL-STYLE POLICY (HUMANEVAL+, 5 SEEDS × 3 FAULTS). ADOPT.: RUNS IN WHICH THE ORACLE LABELS AT LEAST ONE ROOT CONTAMINATED. SPREAD: RUNS WITH AT LEAST ONE CONTAMINATED NON-ROOT REVISION; CONT.: NUMBER OF SUCH REVISIONS. LIVE: NON-ROOT REVISIONS AT THE END OF THE RUN; H SUSP.: REVISIONS SUSPENDED BY HEARTNESS. CLEAN: HELD-OUT PASS@1 OF THE PAIRED CLEAN RUNS.

Generator	Runs	Adopt.	Spread	Cont.	Live	H susp.	Clean
DS-Coder-V2 16B	15	3	2	7	9.0	7.0	0.67
Gemma 2 9B	15	1	0	0	8.0	6.0	0.51
Granite 3.3 8B	15	2	0	0	8.3	6.3	0.64
Llama 3.1 8B	15	5	2	3	9.4	7.2	0.52
Qwen2.5-Coder 7B	15	10	6	6	5.3	3.3	0.71

package. On S2, 30 seeds per configuration are aggregated at the seed level.

C. RQ1: Containment of Faulty Self-Evolution

RQ1: How completely and precisely does closure-based withdrawal contain state affected by a faulty experience, compared with file-level deletion and similarity- and LLM-based detection, and what happens to downstream task behaviour?

We answer RQ1 with three experiments. E-L measures containment where faulty state arises from real LLM generation (S1). E-Q isolates the reachability queries behind containment on structures with exact ground truth (S2), and E-K varies the completeness of capture, the precondition that E-L takes for granted.

1) Experiment E-L: containment in the live harness: Objective. Determine, on state produced by real self-evolution, how much faulty state each remedy removes, how much useful state it removes with it, and whether the remaining harness behaves better. *Procedure.* For every faulty run we computed, after the last evolution task, the suspension set of each remedy from the same final state and the same roots, scored it against the content oracle, and then solved the 29 held-out tasks once per remedy with the post-remedy deployments (alternating D_1 and D_2 contexts, as during evolution; temperature 0). When withdrawal leaves a deployment without a profile, the harness renders the task with that deployment's hand-written base profile, as it does before the first composition; this is the operator action of restoring the base profile that withdrawal itself does not take (Section III-E). The clean run with the same seed provides the reference pass rate.

Results: how often does a faulty lesson take hold? Table IV shows that under this policy a single corrupted feedback message does not reliably contaminate the harness. Across the 75 faulty ExpeL-style runs, the oracle found the planted practice in a root revision in 21 runs (22 if the one ambiguous root label is counted as contaminated) and in at least one further, non-root revision in 10 runs (16 contaminated derivatives in total). Adoption depended strongly on the generator: `qwen2.5-coder:7b` adopted the fault in 10 of its 15 runs and propagated it in 6, whereas `gemma2:9b` and `granite3.3:8b` never propagated it. It also depended on the fault: F1 (print instead of return) was adopted in 5 runs and spread in 1, F2 in 7 and 5, F3 in 9 and 4. Most generators absorbed the corrupted message as one failure among several and wrote insights about other aspects of the batch. Once a fault took hold, however, it persisted: in a typical run (`deepseek-coder-v2:16b`, F1, seed 2) the root insight *"Ensure functions print their final answer with print()."* was re-emitted verbatim by two later merges, the last after task 24, so that D_1 still loaded two copies of it at the end of the run although the root itself had by then been replaced by an unrelated edit. The oracle labelled none of the 210 revisions of the clean runs as contaminated under any of the three fault definitions (630 labels), and the two judges agreed on 99.9% of all 1384 labels ($\kappa = 0.99$, positive agreement 0.99); the only disagreement concerned one root.

Results: containment. Table V compares the remedies. Doing nothing and deleting the roots (R1) leave all 16 contaminated derivatives loadable (residual 0.21 per run), because the faulty advice had already been copied into later insights, typically by the next extraction step. HEARTNESS suspended every contaminated derivative in each of the 10 propagated runs (recall 1.00; better than R1 in 10 of 10 runs, Holm-adjusted $p = 0.014$). Withdrawal without propagation to deployments ($\mathrm{H}_{\mathrm{local}}$) left every contaminated insight in use. This holds by construction, because agents load only what their deployment holds, but it is the practically relevant failure: a store-level deletion does not reach agents that keep loading a deployed copy.

Results: closure equals the time cut in this policy. Containment by closure was expensive: HEARTNESS suspended 5.95 of the 6.00 learned revisions per run (4.90 per propagated run, 3.30 of them clean; precision 0.33, F1 0.48). Its suspension set was *identical* to that of rolling back to the snapshot before the fault (T) in all 75 runs, and almost identical to resetting all learned state (A, which suspended more in 4 runs). Proposition 1 explains why: an ExpeL-style extraction step is shown the whole insight list, so every insight accepted after the root is exposed to it, and the closure reaches the time cut. In this policy, closure-based withdrawal therefore offers no precision advantage over a rollback. What remains is that it is enforced after the fact, refusing later re-admission and unsafe rollback (evidenced in RQ2) and re-derivation from a withdrawn experience (evidenced by unit tests only), none of which the live study exercised, and that it records why each revision was suspended. A third alternative, rolling back and re-evolving without the corrupted experience, corresponds to the paired clean run; it recovers the counterfactual state but requires the post-fault tasks to be replayable. Withdrawing only the faulty root ($\mathrm{H}_{\mathrm{faulty}}$) did not help either: in the propagated runs it suspended the same 4.90 revisions, because the later insights are exposed to every member of the root batch.

Results: detectors. In this policy, content detectors were at least as good as closure. Fig. 2 shows that TF-IDF reached F1 between 0.66 and 0.69 with full recall for every threshold from 0.10 to 0.70, and the embedding baseline did so from 0.70 upwards; at the frozen thresholds they reached 0.60 and 0.50. Contaminated insights repeat the root closely, often verbatim, so the textual signal that similarity needs is present. The LLM judge (R4) was the most precise single detector (precision

TABLE V
CONTAINMENT IN THE LIVE HARNESS, EXPEL-STYLE POLICY. COLUMNS 2–3: REVISIONS SUSPENDED PER RUN, AVERAGED OVER ALL 75 FAULTY RUNS AND OVER THE 10 RUNS WITH PROPAGATED CONTAMINATION. PRECISION, RECALL AND F1: OVER THE 10 PROPAGATED RUNS. RESIDUAL (CONTAMINATED NON-ROOT REVISIONS STILL LOADABLE), COLLATERAL (CLEAN REVISIONS SUSPENDED), PASS@1 AND FAULT-SIGNATURE RATE ON THE HELD-OUT TASKS: OVER ALL 75 RUNS. T, A AND H_{faulty} ARE COMPUTED FROM THE RECORDED STATE AND HAVE NO HELD-OUT EVALUATION (–). MEANS; BOOTSTRAP INTERVALS AND PER-RUN VALUES ARE IN THE REPLICATION PACKAGE.

Remedy	Susp. (all)	Susp. (prop.)	Precision	Recall	F1	Residual	Collateral	Pass@1	Signature
No remedy	0.00	0.00	–	0.00	0.00	0.21	0.00	0.59	0.03
R1 delete root	0.00	0.00	–	0.00	0.00	0.21	0.00	0.59	0.03
T rollback before fault	5.95	4.90	0.33	1.00	0.48	0.00	5.73	–	–
A reset learned state	6.00	5.00	0.33	1.00	0.47	0.00	5.79	–	–
R2 TF-IDF	4.97	4.20	0.50	1.00	0.60	0.00	4.76	0.60	0.02
R3 embedding	6.15	5.30	0.39	1.00	0.50	0.00	5.93	0.61	0.02
R4 LLM judge	2.59	3.70	0.55	0.90	0.65	0.01	2.39	0.60	0.02
HEARTNESS (H)	5.95	4.90	0.33	1.00	0.48	0.00	5.73	0.62	0.02
HS_{judge}	2.36	3.50	0.59	0.90	0.67	0.01	2.16	0.60	0.02
HS_{emb}	4.80	4.10	0.51	1.00	0.62	0.00	4.59	0.61	0.02
$H_{\neg EME}$	5.95	4.90	0.33	1.00	0.48	0.00	5.73	0.62	0.02
$H_{\neg dep}$	5.95	4.90	0.33	1.00	0.48	0.00	5.73	0.62	0.02
H_{local}	0.00	0.00	–	0.00	0.00	0.21	0.00	0.59	0.03
H_{faulty}	1.39	4.90	0.33	1.00	0.48	0.00	1.17	–	–
$H_{EME\text{-}fail}$	5.95	4.90	0.33	1.00	0.48	0.00	5.73	–	–

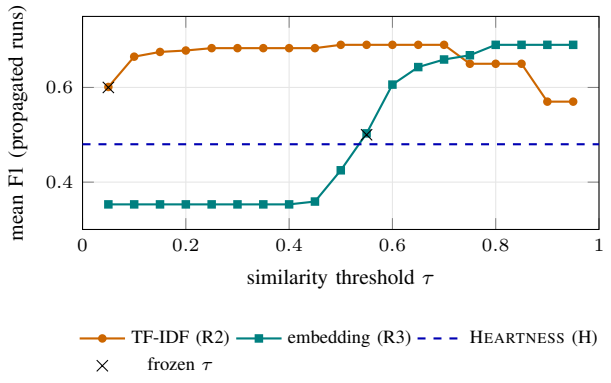


Fig. 2. F1 of the similarity baselines as a function of their threshold on the 10 propagated ExpeL-style runs, against HEARTNESS's F1 (dashed). Crosses mark the thresholds frozen from the pilot. Because contaminated insights repeat the root closely, similarity reaches full recall over a wide range of thresholds.

0.55, F1 0.65) but missed the contaminated revisions of one propagated run (`llama3.1:8b`, F2, seed 1; recall 0.90). HEARTNESS had lower F1 than R4 in 7 and higher in 2 of the 10 propagated runs, and lower than R2 in 6; none of these differences is significant after Holm correction ($p = 0.273$ and $p = 0.352$), which with ten informative runs mainly reflects low power.

Results: closure-bounded revalidation. Revalidation recovers much of the detectors' precision while never looking outside the recorded closure. With the judge as screen (HS_{judge}), 3.50 revisions per propagated run stayed suspended instead of 4.90 (F1 0.67, the highest mean of all conditions at frozen settings), but it inherits the judge's one miss (recall 0.90). With the embedding threshold (HS_{emb}) it kept full recall at F1 0.62 and was never worse than H. Compared with the standalone judge, HS_{judge} differs in 2 of the 10 propagated runs; we therefore do not claim that it outperforms R4, only that the combination never affects anything outside the recorded lineage.

Results: ablations. Removing EME edges or dependency edges changed the suspension set in 0 and 0 of 75 runs. In this policy this holds by construction and is not evidence about the edges: every insight in a run's context is also a source of the next extraction, so each EME edge coincides with a synthesis edge, and insights declare no use dependencies. The curation policy (below) separates the channels.

Results: behaviour. Held-out pass@1 did not discriminate between remedies. Over all runs it was 0.59 without remedy and 0.62 after HEARTNESS, against 0.61 for the paired clean runs and 0.63 for the hand-written harness without any learned state: on these held-out tasks, learned insights did not help, so suspending them could not show a cost. Inference is also not reproducible across processes: held-out prompts that were byte-identical in two different runs, and therefore generated independently, differed between pass and fail in 178 of 4,349 pairs (4.1%). Almost all of these flips (177) occurred on prompts whose context held no learned insight, which recur across all runs of a cell and therefore form most pairs. Within one run, a repeated prompt is answered from the response cache, so the recorded runs replay exactly, but differences of a few items between remedies are within this noise. On the 10 propagated runs, mean pass@1 rose from 0.60 to 0.68 after HEARTNESS, but H was higher in 5 runs and lower in 5 (Holm $p = 1.000$). The fault-signature rate was the more sensitive signal: it fell from 0.09 to 0.02 and was lower after HEARTNESS in 6 of the 10 runs and never higher ($p = 0.219$, not significant). Removing contaminated insights thus removes the planted practice from the agent's code, but in our setting the effect on pass@1 is within inference noise.

Curation policy. The curation policy builds the layered state of the running example, so it tests what the ExpeL-style policy could not: whether closure is narrower than a rollback, whether experience-mediated edges matter, and how detectors

TABLE VI
FAULT ADOPTION AND PROPAGATION PER GENERATOR, CURATION POLICY (HUMANEVAL+, 5 SEEDS × 3 FAULTS). COLUMNS AS IN TABLE IV.

Generator	Runs	Adopt.	Spread	Cont.	Live	H susp.	Clean
DS-Coder-V2 16B	15	6	6	12	29.0	21.4	0.69
Gemma 2 9B	15	13	8	18	29.0	20.9	0.50
Granite 3.3 8B	15	10	8	14	29.0	20.1	0.62
Llama 3.1 8B	15	10	9	14	29.0	20.7	0.48
Qwen2.5-Coder 7B	15	14	11	32	29.0	20.9	0.76

fare when faulty advice is folded into larger artifacts. Tables VI and VII report the 75 faulty curation runs in the same format as before.

Results (curation): adoption and spread. Under curation a single corrupted message took hold far more often. The oracle found the planted practice in the root memory in 53 of 75 runs (59 counting ambiguous root labels) and in at least one further revision in 42 runs, with 90 contaminated derivatives in total. Every generator propagated the fault (Table VI), and so did every fault type (F1 in 7 runs, F2 in 17, F3 in 18). The contamination sat in the composed layers: 42 contaminated skills and 43 contaminated profiles, but only 5 memories. Once the curator had merged the faulty memory into a skill, the next profile composition typically restated it as one rule of the deployment's system prompt, which every later task then loaded (Section II-B quotes such a chain). The oracle labelled none of the 750 revisions of the clean curation runs contaminated (2250 labels, 5 ambiguous); the judges agreed on 99.2% of 4500 labels ($\kappa = 0.89$; positive agreement, the agreement on YES labels, 0.89).

Results (curation): containment and detectors. Doing nothing or deleting the root left 1.20 contaminated derivatives loadable per run. HEARTNESS again suspended every contaminated derivative in all 42 propagated runs (recall 1.00, also 1.00 when ambiguous labels count as contaminated; residual 0.00). Unlike under the ExpeL-style policy, the detectors now missed contamination. The embedding baseline (recall 0.68) missed it in 22 runs, mostly in profiles (28 of its misses), and the LLM judge (recall 0.79) in 13 runs, mostly in skills (12) and profiles (7); HEARTNESS's recall was higher than R3's in 22 runs and R4's in 13, and never lower (Holm $p < 0.001$ and $p = 0.005$). The mechanism is dilution: a skill or profile states the faulty practice in one of five or more rules, so the artifact as a whole is not similar to the root, and the generator, acting as judge, often did not flag such a rule. TF-IDF at the frozen threshold kept recall high (0.99, misses in 2 runs). Its threshold is the lowest of the grid, so it suspends 11.27 revisions per run, at precision 0.24. Fig. 3 shows that no threshold gives both: the oracle-tuned optimum ($\tau_t = 0.15$, F1 0.69) reaches recall 0.75, and the embedding baseline never exceeds F1 0.21. Where the textual signal is diluted, a detector must trade recall for precision, and its thresholds cannot be validated without ground truth. On F1 and collateral the comparison reverses. TF-IDF and the LLM judge had higher F1 than HEARTNESS (the embedding detector was comparable: 0.16 against 0.18, HEARTNESS higher in 27 runs and lower in 14, Holm $p = 0.091$), R2 in all 42 propagated

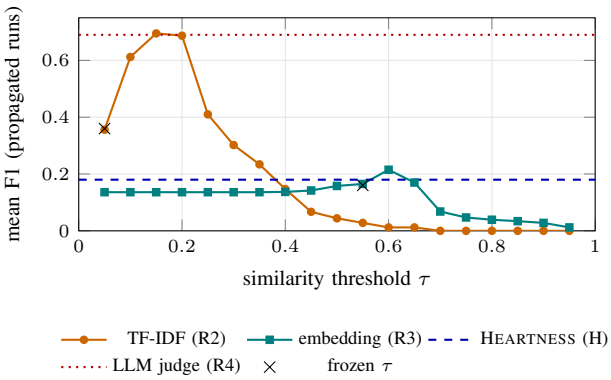


Fig. 3. F1 of the similarity baselines as a function of their threshold on the 42 propagated curation runs, against HEARTNESS (dashed) and the LLM judge (dotted). Unlike under the ExpeL-style policy (Fig. 2), F1 peaks sharply at a low threshold, where TF-IDF recall is 0.75; among the plotted conditions, only HEARTNESS and R2 at the frozen threshold have recall near 1.

runs (Holm $p < 0.001$). R4 had higher F1 than HEARTNESS in 36 of them and lower in 6 ($p < 0.001$), and HEARTNESS suspended more clean revisions than R2 in 73 and than R4 in 75 of 75 runs ($p < 0.001$ and $p < 0.001$), and more than R3 in 41 (fewer in 31; $p = 0.048$). All contamination metrics here have an effective sample of 42 runs.

Results (curation): closure versus the time cut. Here the closure was strictly narrower than a rollback. HEARTNESS suspended 20.79 revisions per run against 22.20 for the time cut T and 29.00 for resetting all learned state; its suspension set was contained in T's in all 75 runs, as Proposition 1 requires, and strictly smaller in 42, with equal recall. The difference is small, and its location explains why. All 106 revisions that T suspends and HEARTNESS spares are memories created between the fault and the first profile composition (after task 12) by tasks that never loaded the faulty memory or anything derived from it. From the composition on, the deployment's profile is in the closure (in 75 of 75 runs), every task loads its profile, and so every later experience, and every memory written from it, is exposed. It is the EME rule applied to a broadcast context that pulls the closure up to the time cut: the precision of closure-based withdrawal depends jointly on how selectively the harness assembles contexts and on which runs count as exposure. The size of the gap is therefore a function of the fault position relative to the first composition: closure was strictly smaller than T in 0 of 15 runs with the fault at task 2, 33 of 45 at task 8 and 9 of 15 at task 9; faults after the first composition (task 12) were not sampled; there the next composition is at task 24, so later tasks would be exposed only through memory retrieval and the gap could be larger. In absolute terms HEARTNESS's precision remained low (0.10, F1 0.18), below every detector's, by a large margin for R2 and R4 and narrowly for R3. Withdrawing only the root that the oracle labels faulty (H_{faulty}) would have missed contamination in 3 runs whose derivatives were labelled contaminated but whose root was not; in all 3 the judges disagreed on the root. An operator who withdraws only revisions that look wrong inherits such misses, which is why H withdraws every revision motivated by the suspicious experience. Restricting attention

TABLE VII
CONTAINMENT IN THE LIVE HARNESS, CURATION POLICY. COLUMNS AS IN TABLE V; SUSP. (PROP.), PRECISION, RECALL AND F1 OVER THE 42 PROPAGATED RUNS, ALL OTHER COLUMNS OVER ALL 75 FAULTY RUNS. $H_{\neg dep}$ EQUALS H BY CONSTRUCTION (EVERY PROFILE DEPENDENCY IS ALSO A SYNTHESIS SOURCE); H_{local} SUSPENDS REVISIONS IN THE STORE BUT CANNOT REMOVE DEPLOYED COPIES, WHICH AGENTS KEEP LOADING. $H_{EME\text{-}fail}$ COUNTS EME EXPOSURE ONLY FOR RUNS WHOSE TASK FAILED.

Remedy	Susp. (all)	Susp. (prop.)	Precision	Recall	F1	Residual	Collateral	Pass@1	Signature
No remedy	0.00	0.00	–	0.00	0.00	1.20	0.00	0.61	0.03
R1 delete root	0.00	0.00	–	0.00	0.00	1.20	0.00	0.61	0.03
T rollback before fault	22.20	22.86	0.10	1.00	0.17	0.00	20.68	–	–
A reset learned state	29.00	29.00	0.07	1.00	0.14	0.00	27.48	–	–
R2 TF-IDF	11.27	10.48	0.24	0.99	0.36	0.03	9.79	0.60	0.02
R3 embedding	17.84	16.31	0.12	0.68	0.16	0.45	16.97	0.61	0.03
R4 LLM judge	3.48	2.79	0.78	0.79	0.69	0.28	2.27	0.61	0.02
HEARTNESS (H)	20.79	21.57	0.10	1.00	0.18	0.00	19.27	0.62	0.02
HS_{judge}	3.35	2.90	0.71	0.79	0.65	0.28	2.13	0.60	0.02
HS_{emb}	13.96	13.52	0.19	0.86	0.27	0.19	12.68	0.61	0.02
$H_{\neg EME}$	14.39	16.19	0.21	0.99	0.31	0.01	12.88	0.61	0.02
$H_{\neg dep}$	20.79	21.57	0.10	1.00	0.18	0.00	19.27	0.62	0.02
H_{local}	15.59	16.29	0.01	0.03	0.01	1.13	15.49	0.61	0.03
H_{faulty}	14.97	20.14	0.10	0.93	0.16	0.15	13.61	–	–
$H_{EME\text{-}fail}$	17.24	18.76	0.13	1.00	0.22	0.00	15.72	–	–

to the 53 runs whose root adopted the fault does not change the picture: HEARTNESS suspended 21.19 revisions per run there, against 22.49 for T and 2.70 for R4.

Results: selectivity beyond the sampled fault positions. Our planted faults fell at tasks 2, 8 and 9, before the first composition. Because closure and the time cut are computed from recorded structure alone, we can compare them for revisions created at any other position without the oracle, assuming that a fault there would leave the recorded exposure structure unchanged: we took every revision active at the end of each of the 200 runs (faulty and clean) as a hypothetical root and compared its closure with the revisions written at later tasks (Table VIII; revisions written in the same step as the root are excluded, since a step-level rollback could also spare them). This measures only how much closure spares, not whether what it spares is clean. Under the ExpeL-style policy closure equalled the time cut for every root: every later extraction step sees the whole list. Under curation, roots created after the first composition had closures covering on average 0.46 (tasks 13–18) and 0.61 (tasks 19–24) of the later revisions (fewer than all for 97% and 95% of them; in absolute terms, 6.2 of 11.9 and 2.5 of 6.0 later revisions spared on average), against 0.94 for roots at tasks 7–12, because a memory written after task 12 reaches later contexts only through retrieval until the next composition. The sparing is a window effect: the profiles composed at task 24 were in the closure of every such root (700 and 500 roots), so after the next broadcast composition the closure would again reach the time cut, which is also why roots at tasks 19–24, whose later revisions include the task-24 profiles and skill, show a higher ratio than those at tasks 13–18. The low ratios therefore partly reflect the 24-task horizon. Counting EME only for failed runs lowered the ratios further (0.63 for tasks 7–12). Closure is thus nearly a time cut for early revisions and exactly one for policies that show every step everything, and markedly more selective for revisions created after the first composition under retrieval, until the next composition.

TABLE VIII
STRUCTURAL SELECTIVITY: MEAN SHARE OF THE REVISIONS WRITTEN AT LATER TASKS THAT FALL IN $\mathrm{Cl}(r)$, OVER EVERY FINAL ACTIVE REVISION r TAKEN AS A HYPOTHETICAL ROOT, BY THE TASK THAT CREATED r, WITH THE SHARE OF ROOTS WHOSE CLOSURE MISSES AT LEAST ONE LATER REVISION (SMALLER). EME-FAIL: EME COUNTED ONLY FOR FAILED RUNS. ORACLE-FREE; ALL 200 RUNS; ROOTS ARE THE REVISIONS STILL ACTIVE AT THE END OF A RUN (INSIGHTS EDITED OR REMOVED EARLIER ARE NOT ROOTS), AND ROOTS WITHOUT LATER REVISIONS ARE OMITTED. THE EXPEL-STYLE 1–6 ROW RESTS ON A SINGLE ROOT.

	ExpeL-style				Curation			
Root task	Roots	Ratio	EME-fail	Smaller	Roots	Ratio	EME-fail	Smaller
1–6	1	1.00	1.00	0%	700	0.92	0.79	43%
7–12	73	1.00	1.00	0%	700	0.94	0.63	56%
13–18	158	1.00	1.00	0%	700	0.46	0.38	97%
19–24	162	1.00	1.00	0%	500	0.61	0.59	95%

Results (curation): which edges matter. Without EME edges, the closure changed in 55 of 75 runs: it suspended 14.39 instead of 20.79 revisions per run (F1 0.31 instead of 0.18) but left contamination loadable in 1 run. That run is the behavioural inheritance of the running example: in `llama3.1:8b`, F3, seed 3, the corrupted feedback produced memory m_8 (*"…use integer division (//) instead of true division (/)…"*); the next task retrieved m_8 into its context, the agent's solution used floor division, and the reflection, shown only two unrelated memories as sources, wrote m_9: *"…consider using the // operator for integer division instead of the / operator…"*. Only the exposure of that run to m_8 connects m_9 to the fault. EME edges therefore bought one contaminated revision in 42 propagated runs at the price of more collateral in 55 runs, on average 20.79 instead of 14.39 suspended revisions per run. EME is a policy of the controller, not a fixed rule: counting exposure only for runs whose task failed ($H_{EME\text{-}fail}$, a post-hoc variant computed from the recorded state) kept recall 1.00 including the run above, whose task had failed, and suspended 17.24 revisions per run (F1 0.22); whether that price is worth paying depends on how

costly a residual faulty lesson is relative to a temporarily suspended clean one, which is a deployment decision, not one our data can settle. Dependency edges changed nothing (0 runs differ) because every profile's dependencies on skills are also its synthesis sources; dependency-only consumers are covered by the synthetic motifs of E-Q. Withdrawal without deployment propagation (H_{local}) left contamination in use in every propagated run (recall 0.03).

Results (curation): revalidation. Closure-bounded revalidation is limited by its screen, and helped by the dependency rule of Algorithm 5. With the judge as screen, HS_{judge} kept 2.90 revisions per propagated run suspended (F1 0.65) but, like the judge alone, restored contaminated revisions in 13 runs (recall 0.79); under curation it differs from the judge alone in 11 of 42 propagated runs but has the same recall and slightly lower F1 (0.69 for R4) because 19 cleared profiles, none of them contaminated, were rejected while a skill they depend on stayed suspended. With the embedding screen the rule mattered: the screen cleared 20 contaminated revisions whose dependencies it did not clear, and admission rejected them (76 rejections in 51 runs), so HS_{emb} reached recall 0.86 against 0.68 for the same screen applied to the whole state (R3). Revalidation never touches anything outside the closure, but under curation it cannot recover the detectors' precision without giving up part of the closure's recall; a screen with higher recall than the generator-as-judge would be needed.

Results (curation): behaviour. Held-out pass@1 was again uninformative: 0.61 without remedy, 0.62 after HEARTNESS, 0.61 for the clean runs and 0.62 for the hand-written harness. On the 42 propagated runs it rose from 0.60 to 0.62 after HEARTNESS (higher in 23 runs, lower in 13, Holm $p = 0.338$), and the fault-signature rate fell from 0.04 to 0.02 (lower in 12 runs, higher in 2, Holm $p = 0.071$). Under both policies the means move in the expected direction, and neither difference is significant. The propagated-run gain in pass@1 corresponds to less than one of 29 held-out items per run. Byte-identical prompts generated independently in two runs differed in 2 of 273 pairs (0.7%) under curation (4.1% under the ExpeL-style policy).

2) Experiment E-Q: lineage queries with exact ground truth: Objective. Separate the accuracy of the queries on which containment rests (origins of a revision, its descendants, and the affected set of a withdrawal) from the accuracy of the content oracle. *Procedure.* On 810 synthetic scenarios (nine structural motifs $\times$ three sizes $\times$ 30 seeds) each method answered all three queries for every revision, scored by F1 against construction-time truth. A separate configuration rewrote 0%, 50% or 100% of each derived revision's text while keeping its lineage.

Results. Recorded exposure answered all queries exactly (impact F1 1.000), within-file history recovered almost none of the cross-file lineage (impact 0.155, descendants 0.108), and lexical recovery was in between (impact 0.699, origins 0.374). Paraphrasing lowered lexical impact F1 from 0.755 (no rewriting) to 0.687 and 0.689 (half and full rewriting) and did not affect recorded exposure. As intended, the capture-only baseline answers the queries exactly as HEARTNESS does (impact 1.000): with complete edges the query is the same reachability computation. What capture alone lacks is shown elsewhere: H_{local} in E-L (no deployment-wide suspension), and RQ2 (no admission or rollback guards). A direct-edge linter that flags revisions with a withdrawn parent, without transitive propagation, reached F1 0.229 on 90 post-withdrawal states with 0 false alarms on clean controls: transitive closure, not detection of direct parents, is what finds the affected set.

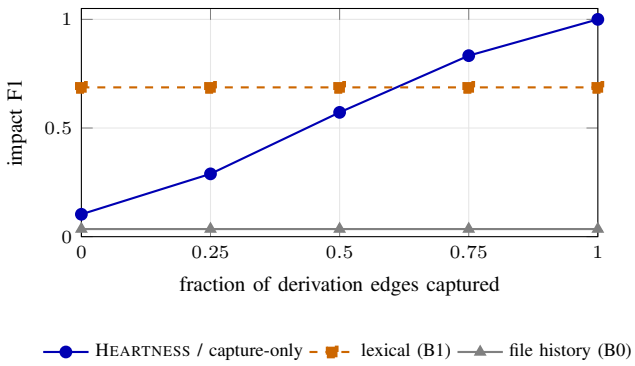


Fig. 4. Impact F1 as capture completeness varies (synthetic benchmark, 30 seeds per point). Recorded exposure degrades in proportion to missing edges; somewhere between 50% and 75% capture, lexical recovery becomes more accurate.

3) Experiment E-K: sensitivity to capture completeness: Objective. Quantify how much containment depends on the completeness of capture (assumption A2). *Procedure.* We hid a random 0–100% of the derivation edges from each method before querying, 30 seeds per level.

Results. Fig. 4 shows that recorded exposure is only as good as its capture: impact F1 fell from 1.000 with complete capture to 0.833, 0.572 and 0.289 with 75%, 50% and 25% of the edges, and to 0.103 with none, whereas lexical recovery, which does not use edges, stayed at 0.687. Somewhere between half and three quarters of the edges, a lexical detector becomes the better tool. This is the quantitative reason for host-side capture (Section III-C): the controller, not the policy, must record exposure, and it must do so for every input it supplies.

Answer to RQ1. Closure-based withdrawal had recall 1.00 in all 52 propagated runs, as did rollback, reset and the EME-fail variant. What it suspends is bounded by the time cut: the same set under the ExpeL-style policy, slightly less under curation. Every detector was more precise, and under curation TF-IDF and the LLM judge had significantly higher F1, but detectors missed contamination folded into skills and profiles (R3: 22, R4: 12, R2: 1 of 42 runs). Structurally, closure is markedly more selective than a time cut only for curation revisions created after the first composition, and only until the next composition. EME edges were necessary once, at a large collateral cost. Effects on held-out behaviour were not significant.

D. RQ2: Assurance of the Lifecycle Guards

RQ2: Do the lifecycle guards prevent property violations that their removal would allow, and does bounded model checking find such violations with witnesses that hold on the implementation?

1) Experiment E-G: targeted guard challenges: Objective. Establish, guard by guard, that each guard or audit predicate is what prevents or exposes a specific violation, rather than assuming it. *Procedure.* For each property P1–P10 we construct one challenge on the running example that targets it: an inadmissible event for guard-enforced properties (P1–P4, P6, P8, P9), and out-of-API corruption of the stored state for the audit-only properties (P5, P7, P10). Each challenge runs twice: on the full engine and on an engine with the targeted guard (or audit predicate) removed.

Results. With all guards present, every guard-enforced challenge was either rejected (P1, P3, P4, P6, P9) or contained without any violation (P2, P8: suspension removes the affected revisions); the out-of-API corruptions were flagged by the audit (P5, P7, P10). Removing the targeted guard or predicate changed the outcome of 10 of the 10 challenges to the targeted failure: the event was admitted and the audit reported the targeted violation, or the corruption went unreported. Removing the withdrawal guard (the P2 suspension) also produced a P8 violation, because the same suspension protects deployments. The result is close to definitional, and the mutation tests (Section IV-A) already show that the test suite detects removed guards; what E-G adds is a property-by-property map from each guard to the concrete violation it prevents on the running example, which the bounded checks of E-M then generalize over all short operation sequences.

2) Experiment E-M: bounded model checking and witness replay: Objective. Check whether the guard semantics admit any violating operation sequence within a bound, and whether violations found on the model also occur on the implementation. *Procedure.* For three identities, the three topologies and a depth bound of 10 operations, we ran the reference projection and five guard mutants (P2, P3, P4, P8, P9 removed) through both the Python BFS and SPIN (18 configurations), replayed every BFS witness on the projection and on the engine with the same guard disabled, and re-ran BFS at depth $\ell - 1$ to check minimality. A separate experiment (E-C) compared engine and projection on every successor of all reference states reachable within depth 6 for two identities, under an explicit state mapping.

Results. Table IX lists all 18 configurations. BFS and SPIN agreed on 18/18 verdicts. The reference guards were safe within the bound in all topologies (up to 93,266 states). 13 mutant configurations violated a property, with shortest witnesses of 2–7 operations; all 13 witnesses replayed on the engine with the same guard removed and produced the targeted audit violation, and all 13 were minimal (no violation at depth $\ell-1$). The 2 remaining mutant configurations (P2 and P3 removed, independent topology) were safe, as expected: without edges there is nothing for those guards to protect. They act as negative controls showing that the checker does not report violations indiscriminately. In E-C, 3,000 attempted transitions (992 enabled) were compared and 0 mismatches were found.

These checks also paid off during development, which is a qualitative result in its own right. In an earlier campaign (retained in the replication package), replaying model-checking witnesses on the engine failed for four witnesses. The cause was a missing guard in the abstract specification (proposals could cite a source that had not been installed), shared by both encodings, and it was invisible to verdict agreement alone. A second discrepancy, found by mapped-state comparison, concerned whether DEPLOY replaces or extends deployment membership. Both were fixed in the specification and the implementation, and all reported campaigns use the corrected versions.

TABLE IX
BOUNDED MODEL CHECKING (3 IDENTITIES, DEPTH 10). STATES: BFS STATES EXPLORED. SPIN S: SPIN VERIFICATION TIME. VERDICT: BFS VERDICT AND SHORTEST WITNESS LENGTH; AGREE: SPIN VERDICT EQUALS BFS VERDICT.

Guards	Topology	States	SPIN s	Verdict	Agree
reference	chain	5,230	1.28	safe in bound	yes
reference	independent	93,266	1.10	safe in bound	yes
reference	star	17,649	1.06	safe in bound	yes
P2	chain	140	1.04	violation (7 ops)	yes
P2	independent	93,266	1.08	safe in bound	yes
P2	star	225	1.05	violation (7 ops)	yes
P3	chain	195	1.04	violation (7 ops)	yes
P3	independent	93,266	1.07	safe in bound	yes
P3	star	379	1.07	violation (7 ops)	yes
P4	chain	4	1.11	violation (2 ops)	yes
P4	independent	8	1.02	violation (2 ops)	yes
P4	star	4	1.13	violation (2 ops)	yes
P8	chain	35	1.04	violation (5 ops)	yes
P8	independent	236	1.06	violation (5 ops)	yes
P8	star	43	1.07	violation (5 ops)	yes
P9	chain	19	1.14	violation (4 ops)	yes
P9	independent	82	1.12	violation (4 ops)	yes
P9	star	20	1.05	violation (4 ops)	yes

TABLE X
GUARD FAULTS DETECTED OUT OF 30 TRIALS UNDER EQUAL TIME BUDGETS (2 IDENTITIES, DEPTH 9).

Guard removed	Random	Hypothesis	BFS
reference	0/30	0/30	0/30
P2	1/30	0/30	30/30
P3	0/30	0/30	30/30
P4	30/30	27/30	30/30
P8	25/30	1/30	30/30
P9	30/30	3/30	30/30

3) Experiment E-S: exhaustive versus sampled exploration: Objective. Assess whether exhaustive bounded exploration is needed, or whether cheaper randomized or property-based testing finds the same guard faults. *Procedure.* For two identities and depth 9, 30 trials per mutant ran uniform random sequences and Hypothesis-generated sequences under the same wall-clock budget as BFS.

Results. Table X shows that BFS detected every seeded fault in every trial and raised no false alarm on the reference guards. Random sequences found the faults that need few, unconstrained steps (P4, P9, and P8 in most trials) but rarely or never found those that need a specific ordering of acceptance, withdrawal and re-acceptance (P2, P3); Hypothesis did worse on most faults under the same budget. The faults that sampling misses are precisely those concerning withdrawn lineage.

Discussion. The guards are individually necessary for their properties, exhaustive bounded exploration is necessary to find

ordering-dependent faults reliably, and witness replay plus mapped-state comparison catch model–implementation mismatches that verdict agreement does not. The evidence remains bounded: three identities, depth 10, a projection without text, effects or EME, and with contracts reduced to the presence of dependencies.

Answer to RQ2. Each guard prevents its targeted violation (10/10 targeted challenges change outcome when it is removed). Within the checked bounds the reference lifecycle is safe; BFS and SPIN agree on all 18 configurations; all 13 witnesses are minimal and reproduce on the implementation; and, on seeded guard faults at small scope, exhaustive exploration finds ordering-dependent withdrawal faults that equal-budget random and property-based testing miss.

E. RQ3: Cost and Limits

RQ3: What do capture, admission, withdrawal and checking cost, and what can runtime conformance monitoring detect or not detect?

1) Experiment E-O: overhead in the live harness: Objective. Measure the controller's share of the cost of real self-evolution. *Procedure.* In every live run we timed all controller calls (capture, admission, snapshots, LOAD) separately from LLM generation and test execution, and timed each remedy.

Results. The controller is a negligible part of live self-evolution. In the ExpeL-style runs, one run took 3.9 minutes of evolution on average (198 LLM calls, about 50k input and 21k output tokens), of which all controller operations together (capture, admission, snapshots, LOAD and deployment updates, about 106 history events) took 0.49 s, or 0.21% of the evolution time. Computing a withdrawal plan with HEARTNESS took 20.3 ms per run, against 1.1 ms for TF-IDF, 3014.3 ms for the embedding baseline (dominated by embedding calls) and 20614.6 ms for the LLM judge, which must read every live revision. Under curation, contexts and state are larger (6.1 minutes, 305 LLM calls and about 156 history events per run); the controller took 1.71 s, or 0.47% of evolution time, and a withdrawal plan 29.3 ms against 66236.9 ms for the LLM judge. Revalidation adds judge or embedding calls only for members of the closure.

2) Experiment E-T: scaling of controller operations: Objective. Separate the cost of graph queries from that of full transactional admission and replay as state grows. *Procedure.* Closure queries on synthetic graphs of up to 2,048 revisions (30 seeds); complete guarded workloads of 4–32 artifacts (30 seeds each) measuring capture, admission, withdrawal and monitor replay under Python allocation tracing; and the BFS state space of the projection for 2–6 identities.

Results. A closure query over 2,048 revisions took 28.14 ms on average (1055 KB peak). Full guarded workloads are far more expensive: with 32 artifacts and about 105 history events, admission of the whole workload took 1134.8 ms, withdrawal 22.7 ms, and replaying the history in the monitor 2967.7 ms, versus 28.0 ms, 2.7 ms and 48.1 ms for 4 artifacts. Admission and replay grow super-linearly because the prototype deep-copies state for atomicity and recomputes closures on every acceptance. Table XI shows how quickly exhaustive exploration of the full engine grows with the number of identities: with independent revisions it hits the 20,000-state cap from three identities on, while a chain stays below 6,100 states up to six identities because its dependencies forbid most interleavings. This is why the model checking of RQ2 uses the finite projection rather than the engine.

TABLE XI
STATE-SPACE GROWTH OF EXHAUSTIVE EXPLORATION OVER THE FULL ENGINE (DEPTH 10, AT MOST 20,000 STATES). n: REVISION IDENTITIES; STATES: REACHABLE ENGINE STATES EXPLORED; TIME: WALL-CLOCK SECONDS; COMPLETE: WHETHER EXPLORATION FINISHED BELOW THE CAP.

n	Topology	States	Time (s)	Complete
2	chain	3,317	0.26	yes
3	chain	5,230	0.56	yes
4	chain	5,792	0.82	yes
5	chain	5,971	1.10	yes
6	chain	6,016	1.39	yes
2	independent	9,390	0.77	yes
3	independent	20,000	2.11	capped
4	independent	20,000	2.31	capped
5	independent	20,000	2.56	capped
6	independent	20,000	2.80	capped

3) Experiment E-R: what conformance monitoring detects: Objective. Determine which deviations between logged events and real state the monitor detects. *Procedure.* On the running example, 30 seeded positions each for five scenarios: benign logs, observed-state drift (a revision edited outside the API), invalid events, omitted events, and hidden external actions that are never logged.

Results. The monitor reported no deviation on 0/30 benign logs and detected 30/30 drifts, 30/30 invalid events and 29/30 omitted events. It detected 0/30 hidden external actions: an action that is never logged leaves no trace for a log-based monitor. Across 30 withdrawal-and-rollback scenarios, 30 rollbacks into withdrawn lineage were rejected and external effects were retained in all 30. Replaying the 12 Voyager skill-library revisions (36 events) through the controller produced no conformance finding, showing that the metamodel accepts a real third-party evolution history (this replay does not reconstruct how the skills were synthesized).

Answer to RQ3. In live self-evolution the controller costs 0.21% (ExpeL-style) and 0.47% (curation) of evolution time, and a withdrawal plan takes milliseconds where an LLM-judge screening takes tens of seconds. Closure queries are cheap even for thousands of revisions, but full transactional admission and replay in our prototype grow super-linearly and reach 1–3 s for 32-artifact workloads; finite checking scales only to a handful of identities. The monitor detects drift, invalid events and nearly all omissions, and by construction cannot detect actions that are never logged.

F. Threats to Validity

Construct validity. Our ground truth for contamination is a content oracle of two LLM judges, not human annotation. We mitigate this by using judges from families different from every generator, schema-constrained answers, unanimity, reporting ambiguous labels and their effect, measuring agreement (κ), checking the judges' false-positive rate on clean runs,

and complementing content labels with behavioural measures (HumanEval+ tests and static fault signatures). A judge can still miss a paraphrase or flag an artifact that merely mentions the practice. Exposure, which HEARTNESS records, is not influence; we therefore never evaluate HEARTNESS against its own records but against content and behaviour. Pass@1 on held-out tasks measures one aspect of harness quality and is sensitive to which state remains loadable; a remedy that removes useful state is penalized only insofar as that state helped on those tasks. In the synthetic benchmark, ground truth is exact by construction but its structures are generated, and with complete capture HEARTNESS and capture-only provenance answer queries identically by design.

Internal validity. The faults are planted by corrupting one feedback message; real faulty lessons can arise more gradually or from several experiences. The fault position took only three values (tasks 2, 8, 9) for our seeds; early faults favour propagation and inflate closure sizes relative to late faults. Revalidation and the post-hoc conditions (T, A, H_{faulty}) were designed after the local pilot, and $H_{\text{EME-fail}}$ after the first analysis of the reported campaign, when the single run that only EME edges trace was known to have failed; its recall therefore rests on that run. The structural selectivity analysis was added late, in response to internal review of an earlier version of this paper, and its definition was revised once: a first version that counted revisions written in the same step as the root made the ExpeL-style policy look selective for late roots, which it is not. Revalidation was re-run once on the curation records with the dependency rule of Algorithm 5, after a review found that the first evaluation had not applied it; the superseded results are kept in the package; revalidation was evaluated on the recorded state in a separate process rather than through the controller's admission operators. Thresholds of the similarity baselines were tuned on a local pilot with a different generator (`qwen2.5:7b`) and the curation policy and transferred unchanged to all generators and both policies; tuning per model and policy could improve them, which we bound with an oracle-tuned upper bound. The R4 judge is the generator itself, as a harness would use it; a stronger external judge could detect more, but would then be close to the oracle. LLM decoding used temperature 0 and fixed seeds, but batched GPU inference is not reproducible across processes: byte-identical prompts generated independently in two runs flipped pass/fail in 4.1% (ExpeL-style) and 0.7% (curation) of pairs. On 2 of the 25 oracle jobs of the curation campaign the inference server fell back to CPU because GPU discovery timed out; those judges ran at temperature 0 and their labels were kept. Generation jobs whose server fell back to CPU failed, found their cells already complete, or (after we added a device check to the job script) restarted the server until it ran on GPU, so every reported evolution run and revalidation was generated on GPU (`inference_devices.tsv` in the package lists the device of every job). Every response is cached, so the reported runs replay exactly, while a fresh run reproduces the protocol rather than the exact texts. The harness, the remedies and the oracle prompts were written by us; we report all prompts in the replication package.

Platform validity. The live study ran on one cluster and one GPU type; we did not measure how much results vary across hardware, because the smoke runs on other sites were too short (8 tasks, one seed) to say. All compared conditions share the platform, which protects the comparisons but not the absolute numbers.

External validity. The live harness is small (24 evolution tasks, two deployments) compared with production agents, and the tasks are function-level Python problems; the two harness policies (curation into skills and profiles, and ExpeL-style insight lists) are two of many update mechanisms [6]. We used five open-weight 7–16B generators; larger proprietary models may follow or filter faulty lessons differently. Capture completeness holds only for inputs the host mediates (assumption A2); production harnesses with tool results, web content or shared memories across users would need corresponding instrumentation. The synthetic benchmark and the bounded model checking cover small graphs and few identities.

Conclusion validity. Each faulty run is a paired observation across remedies; we use non-parametric paired tests, effect sizes and Holm correction, and ship bootstrap confidence intervals with the per-run values. Runs in which the generator did not adopt the planted fault, or in which it did not propagate beyond the root, carry no information about recall; we report how many there were and compute precision, recall and F1 only on the others, which reduces the effective sample size for those metrics to 10 of 75 ExpeL-style and 42 of 75 curation runs. We named no single primary metric; recall and residual contamination are the metrics the approach is designed for, and we report all others with equal prominence, including those that favour the detectors. Differences that are consistent in direction but not significant after correction are reported as such.

G. Lessons Learned and Limitations

Report adoption, not only containment. A corrupted feedback message became a contaminated revision in only part of the faulty runs, and how often depended strongly on the generator, the fault and the policy: propagation occurred in 10 of 75 ExpeL-style runs but 42 of 75 curation runs (Tables IV and VI). A containment study that plants faults and reports only averages over all runs would mostly measure runs with nothing to contain. We therefore report adoption and propagation counts, compute recall only where there was something to recall, and recommend the same for future studies of faulty or poisoned self-evolution.

The policy decides what closure costs. Closure suspends what was *exposed*, and how much is exposed is a design decision of the self-evolving policy. By Proposition 1, withdrawal never suspends more than a rollback to before the fault, and in our ExpeL-style runs, where every extraction step sees the whole insight list, it suspended exactly as much. Under curation it was smaller only until the first composed profile was deployed; from then on every task loaded it, and EME edges made every later lesson exposed. Conversely, for roots created after that composition, which reach later contexts only through retrieval, closures covered on average 0.46–0.61 of the later state (Table VIII), but only until the next broadcast

composition. Harness designers who want withdrawal to be cheaper than a rollback must give each generation step and each run only the inputs it needs, because the host records exactly what it supplies, and can restrict which runs count as exposure: counting only failed runs, a post-hoc variant, cut suspensions from 20.79 to 17.24 per run and kept the one EME-only contamination in our data, which arose from a failed run; whether lessons from passing runs inherit faults is a question for future data. This is the trade-off of narrow interfaces in modular design, now measured on generated state.

Closure is a bound, detection is a filter. Content detectors were more precise than closure throughout, and lost recall where synthesis folded contaminated advice into larger artifacts: under curation the embedding detector and the LLM judge missed a contaminated skill or profile in 22 and 12 of 42 propagated runs. Using the closure as the candidate set and a detector only to re-admit members bounds what the detector can touch and recovers precision, but it also re-admits whatever the detector misses: under curation, judge-screened revalidation restored contaminated revisions in 13 runs. The recorded dependencies help: requiring a re-admitted profile's skills to be re-admitted first kept 20 contaminated revisions out that the embedding screen had cleared. Closure gives recall, detection gives precision, and with today's screens a maintainer has to choose which error is cheaper for the deployment at hand.

Withdrawal must reach deployments. In a harness that gives each deployment its own loaded copy, a withdrawal that changes the main store but not the deployments has no effect on behaviour (H_{local} cannot remove deployed copies). This is a design requirement rather than a finding: multi-deployment harnesses need deployment membership as a first-class, recorded relation, and any remedy, including a rollback, must be redeployed to take effect.

Measure behaviour with signatures, not only pass rates. Held-out pass@1 was insensitive in our setting: learned state did not improve it, and independent generations of the same prompt differed on up to 4.1% of items, as much as the differences between remedies. Fault signatures, static checks for the planted practice, decreased after containment under both policies while pass@1 moved in both directions, although neither change was significant. Studies of harmful self-evolution should report a behavioural measure specific to the harm and measure inference noise on independently generated, identical prompts.

Check the model against the code, not only against another model. Two independent encodings agreed on a specification that had a gap (Section IV-D); only replaying witnesses on the implementation exposed it. Writing the inductive argument of Proposition 2 exposed another: an experience could name a run whose context was recorded only later, which would have added exposure edges to revisions already accepted; the engine now rejects such a LOAD. We consider witness replay, mapped-state comparison and explicit invariance arguments complementary parts of using model checking for an evolving implementation.

Make live studies replayable. Caching every model response by a digest of model, prompt and options made each live run replayable without GPUs and decoupled analysis from inference; end-to-end smoke runs of the whole batch pipeline found several deployment faults before any reported run, and checking the inference device of every job revealed that the model server silently fell back to CPU on some GPU slices.

Limitations. Exposure is not influence, so withdrawal over-approximates by design; the ground truth is a two-judge content oracle, not human annotation; faults are planted and single; the live harness is small and its tasks are function-level; capture is complete only for inputs the host mediates and cannot see unlogged actions; quarantine is permanent in our prototype, so a corrected skill returns only through revalidation or fresh lineage; withdrawal cannot prevent a policy from re-learning a faulty lesson from *new* evidence; and the model checking is bounded and covers a projection without text, effects or experience-mediated edges. We did not study usefulness for human maintainers; a natural study would give maintainers incident-response tasks on recorded harnesses with and without withdrawal plans and measure time to containment and residual contamination. The live study withdrew only after the last evolution task, so the enforcement that distinguishes withdrawal from a one-off rollback (refused re-admission and unsafe rollback, evidenced by the controlled scenarios of RQ2, and refused re-derivation, evidenced by unit tests only) was not exercised by live evolution.

V. RELATED WORK

We group related work into six areas and, for each, state what HEARTNESS adds and where the work is complementary. Table XII summarizes the comparison.

A. Self-Evolving Agents and Harness Learning

Reflexion [1], ExpeL [2], Agent Workflow Memory [3] and generative agents [38] accumulate natural-language memory or workflows; Voyager [4] grows an executable skill library; ADAS [39], DSPy [40] and the Darwin Gödel Machine [5] search over prompts, pipelines or agent code. These systems are the *producers* of the persistent state that HEARTNESS governs. They evaluate whether evolution improves task scores, and the survey of Wei et al. explicitly separates such improvement claims from records of what state operated and identifies provenance across synthesis and reuse, and withdrawal of derivatives, as open problems [6]. None of these systems records which stored artifacts a generated artifact was exposed to, or offers withdrawal with deployment-wide containment. HEARTNESS is complementary: our live harness is a deliberately simple member of this family, and the controller is policy-independent.

B. Memory and Skill Poisoning; Runtime Enforcement

AgentPoison shows that a small number of poisoned memory or knowledge-base entries can steer agents [13], and indirect prompt injection shows that retrieved content can carry instructions [41]. EVOMAL shows that self-evolving coding agents copy malicious code from retrieved skills into newly authored ones, and that the threat persists after the

planted skills are removed; it proposes a counter-prompt and a quarantine gate for new skills [8]. Our study observes the same persistence with a *non-adversarial* faulty lesson and evaluates a candidate remedy for the non-adversarial case: identifying and suspending derivatives by recorded exposure rather than by detecting the payload. We did not evaluate adversarial or executable-skill poisoning. AgentSpec enforces user-written rules on agent actions at runtime [12], and VASO model checks skill contracts and turns counterexamples into skill revisions [15]. These approaches constrain what an individual skill or action may do; HEARTNESS constrains how persistent state may change and be undone, and could host either as a validator in its admission step.

C. Provenance Capture

PROV-DM and the Open Provenance Model standardize entities, activities and derivations [10], [17]; noWorkflow captures provenance of scripts [42]; PROV-AGENT extends PROV to agent interactions, model calls and tool invocations in agentic workflows [11]. Software supply-chain work records build provenance (in-toto [43]) and bills of materials, whose practical adoption challenges Stalnaker et al. document [44]. TraceFL traces a federated model's predictions back to client contributions [45]. Provenance capture is necessary for HEARTNESS and, with complete edges, a capture-only system answers the same reachability queries (our "capture-only" baseline ties HEARTNESS on query accuracy by construction). What capture alone does not give is (i) the recording of every run's context as an input, without which the experience-mediated path is absent from the record even though PROV could express it, (ii) admission that refuses descendants of withdrawn state, (iii) deployment-wide suspension and rollback that cannot reintroduce withdrawn lineage, and (iv) checked lifecycle semantics.

D. Traceability Recovery and Change Impact Analysis

Information-retrieval traceability recovery links artifacts by textual similarity [9]; the field's roadmap emphasizes trustworthy, purposed traceability [23]. Scenario-driven trace analysis derives dependencies from executions [24], and change impact analysis and slicing compute what a change may affect [18], [19]. Our TF-IDF and embedding baselines instantiate IR recovery, and our LLM-judge baseline instantiates semantic detection. The live study shows both sides: when contaminated state repeats its root closely, as in our ExpeL-style runs, similarity recovers it well; when synthesis paraphrases, as in the synthetic paraphrase experiment, lexical recovery degrades while recorded exposure does not. Recorded exposure plays the role that execution traces play in Egyed's approach, and detectors are best used inside the recorded closure (Section III-F).

E. Information Flow, Deletion Propagation, and Invalidation

Experience-mediated exposure is a form of implicit flow. Dynamic taint tracking propagates labels from sources to everything computed from them, as TaintDroid does for private data on smartphones [46]. For LLM agents, CaMeL separates control from data flow and attaches capabilities to values so that untrusted content cannot steer tool calls [47], and FIDES attaches confidentiality and integrity labels to everything an agent planner sees [48]. These systems decide, *within one task*, what a tainted value may influence; HEARTNESS records, *across tasks and deployments*, which persistent state a tainted input may have influenced, and supports withdrawing it afterwards. Integrity labels as in FIDES could serve as an admission validator in HEARTNESS, and HEARTNESS's closure is the persistent-state counterpart of taint propagation, including its well-known over-tainting. In databases, deletion propagation asks which source tuples to delete to remove a view tuple and relates it to why- and where-provenance [49]; our withdrawal propagates in the opposite, forward direction, from a faulty source to everything derived from it. That forward direction is the subject of incremental view maintenance under deletion, whose counting algorithm keeps a derived tuple as long as one derivation survives [50], and of provenance semirings, which distinguish a derivation that requires all of its sources from one that has alternatives [51]. Such fine-grained semantics need to know how an output depends on its inputs. An LLM synthesis step does not reveal whether its output would have been the same without one of its inputs, so HEARTNESS must treat every recorded input as possibly necessary; this is the source of the over-approximation in Section III-E, and the reason closure-bounded revalidation rechecks content instead of re-counting derivations. Machine unlearning removes the influence of training data from model weights, exactly by retraining on the remaining data or approximately [52]. The exact counterpart for a harness is *rollback and re-evolve*, replaying the post-fault experiences without the corrupted one; in our study this is the paired clean run. HEARTNESS instead suspends without recomputation, which is what remains possible when tasks cannot be replayed, experiences are user-specific, or effects have already left the system. Build systems invalidate every target that transitively depends on a changed input [53], and ML model registries record which data and code produced a model [54]; both rely on declared dependencies, which LLM-synthesized state does not declare unless a host records them. Finally, the external effects that rollback cannot undo correspond to the compensation problem of long-lived transactions (sagas [55]); HEARTNESS records such effects but does not compensate them.

F. Model-Driven Engineering and Formal Verification of Evolving Systems

MDE treats models and transformations as primary artifacts [21], [22]; consistency checking [25] and static fault localization in model transformations [56] reason about their correctness. RADIANT generates traceable system models for multi-agent LLM systems and verifies behaviour models with CSP model checking [14], and continuous-assurance work regenerates traceable assurance arguments as specifications and verification results evolve [57]. Alloy [58] and SPIN [27] support lightweight and explicit-state analysis; inductive validity

TABLE XII
CAPABILITY COMPARISON. ✓: PROVIDED; ○: PARTIALLY OR BY AN EXTERNAL COMPONENT; –: NOT ADDRESSED. *Persistent state*: GOVERNS STATE THAT LATER RUNS LOAD. *Synthesis provenance*: RECORDS WHICH STORED ARTIFACTS A GENERATED ARTIFACT WAS DERIVED FROM. *Behaviour flow*: TRACKS INFLUENCE THAT PASSES THROUGH WHAT AN AGENT WAS SHOWN OR DID (TAINT, EXPERIENCE-MEDIATED EXPOSURE). *Admission gate*: CHECKS STATE OR ACTIONS BEFORE THEY TAKE EFFECT. *Withdrawal*: SUSPENDS OR REMOVES EVERYTHING DERIVED FROM A FAULTY ITEM, IN EVERY PLACE IT IS USED. *Rollback*: DEFINES ROLLBACK CONSISTENTLY WITH WITHDRAWAL. *Checking*: MODEL CHECKING OF THE LIFECYCLE OR PLANS. *Runtime*: RUNTIME ENFORCEMENT OR CONFORMANCE MONITORING. *Live eval.*: EVALUATED ON AN EXECUTING SELF-EVOLVING HARNESS.

Approach	Persistent state	Synthesis provenance	Behaviour flow	Admission gate	Withdrawal	Rollback	Checking	Runtime	Live eval.
ExpeL, AWM, Voyager, DGM [2]–[5]	✓	–	–	–	–	–	–	–	✓
Version control (file-level revert)	✓	–	–	–	–	○ (exact, unchecked)	–	–	–
EVOMAL defences [8]	✓	–	–	✓ (skill quarantine)	–	–	–	–	✓
AgentSpec [12]	–	–	–	✓ (actions)	–	–	–	✓	–
CaMeL, FIDES [47], [48]	–	–	✓ (within a task)	✓ (tool calls)	–	–	–	✓	–
VASO [15]	✓	–	–	○ (skill contracts)	–	–	○ (skill plans)	–	✓
PROV-AGENT [11]	–	✓	–	–	–	–	–	–	○
IR traceability recovery [9]	–	○ (inferred)	–	–	–	–	–	–	–
Build systems, model registries [53], [54]	○	✓ (declared)	–	–	○ (invalidation)	–	–	–	–
RADIANT [14]	–	✓ (design)	–	–	–	–	✓ (CSP)	–	–
HEARTNESS (this paper)	✓	✓	✓ (recorded run contexts)	✓	✓ (recorded lineage)	✓	○ (finite projection)	✓	✓

cores show the value of separating soundness, minimality and cost when reporting verification evidence [59]; runtime verification checks executions against specifications [28]. These works target design-time models of a system. HEARTNESS applies the same discipline to state that the running system rewrites itself, and evaluates the model-to-implementation gap explicitly through witness replay, mapped-state correspondence and conformance monitoring.

VI. CONCLUSION

Self-evolving harnesses turn every lesson an agent learns into software that later runs depend on. We presented HEARTNESS, which treats that state as a versioned model: immutable revisions, host-recorded exposure including the experience-mediated path, guarded admission, withdrawal of revisions or experiences with their closure across deployments, revalidation under attestations, and rollback that cannot reactivate withdrawn lineage. We proved that withdrawal is bounded by a time cut, argued that the suspension invariant holds under all operators, checked a finite projection with two independent encodings, and replayed every witness on the implementation.

The live study gives a nuanced answer to whether this machinery is worth it. Faulty lessons do spread, far more under layered curation than in flat insight lists. Closure-based withdrawal, like rollback and reset, removed every contaminated derivative under both policies, and it did so while recording why each revision was suspended. But for the planted faults closure was nearly as conservative as a rollback, because both of our policies broadcast synthesized state into every later context and experience-mediated edges then reach almost everything learned afterwards; structurally, it becomes markedly more selective only for revisions created after the first composition in a retrieval-based harness, until the next composition. Content detectors were more precise throughout and lost recall where synthesis folded the faulty advice into larger artifacts. The two are complementary, and closure-bounded revalidation combines them, with the screen's recall as its limit.

Four directions follow. First, harness designs that assemble contexts selectively, rather than broadcasting composed profiles, and EME policies that count only failed runs (a variant we tested only post hoc), could make closures more precise; our data suggest that both are design choices, not properties of containment. Second, the enforcement that distinguishes withdrawal from a one-off rollback, refusing re-admission, re-derivation and unsafe rollback after the fact, was exercised only in controlled scenarios; a study that withdraws in the middle of live evolution would measure how often it matters. Third, locating the root of a misbehaviour from symptoms, which we assumed given, is the natural complement to containment. Fourth, screens with higher recall than a generator acting as its own judge are needed before revalidation can restore precision without giving up containment.

DATA AND ARTIFACT AVAILABILITY

The replication package contains the HEARTNESS implementation (`src/heartness`), the live self-evolving harness and its HPC pipeline (`experiments/live`, `hpc/`), the synthetic benchmark generator and datasheet (`benchmark/`), the Promela generator and SPIN build script, all raw records of every campaign including superseded ones, the cached responses of every LLM call (so each live run can be replayed without a model server), the processed results, and the script that generates every number in this paper from them (`scripts/paper_macros.py`, with an index from each number to its source file). The live study uses pinned model tags whose digests are stored in every record and pinned EvalPlus dataset revisions with checksums. `README.md` gives one-command reproduction for each subject; for the live study, `make live-fir` re-derives every live table and figure from the shipped raw records and caches without a GPU, and `hpc/` contains the batch pipeline that produced them. An archival DOI will be minted for the camera-ready version.

REFERENCES

[1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[2] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642.

[3] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent workflow memory," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 63 897–63 911.

[4] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024.

[5] J. Zhang, S. Hu, C. Lu, R. T. Lange, and J. Clune, "Darwin Gödel machine: Open-ended evolution of self-improving agents," in *The Fourteenth International Conference on Learning Representations (ICLR)*, 2026.

[6] C. Wei, H. Guan, M. Zhu, K. Zhou, J. He, X. Zhou, H. Chen, and Y. Shen, "Harness self-evolution: Theory, algorithms, evaluation, applications, and impact," arXiv:2610.18813, 2026. [Online]. Available: https://arxiv.org/abs/2610.18813

[7] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[8] X. Wu, Y. Shi, Q. Li, Z. Zhao, X. Li, B. Adams, A. E. Hassan, and J. Ni, "EVOMAL: Self-poisoning in self-evolving coding agents," arXiv:2608.25776, 2026. [Online]. Available: https://arxiv.org/abs/2608.25776

[9] G. Antoniol, G. Canfora, G. Casazza, A. D. Lucia, and E. Merlo, "Recovering traceability links between code and documentation," *IEEE Transactions on Software Engineering*, vol. 28, no. 10, pp. 970–983, 2002.

[10] "PROV-DM: The PROV data model," W3C Recommendation, 2013. [Online]. Available: https://www.w3.org/TR/2013/REC-prov-dm-20130430/

[11] R. Souza, A. Gueroudji, S. DeWitt, D. Rosendo, T. Ghosal, R. Ross, P. Balaprakash, and R. F. da Silva, "PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows," arXiv:2508.02866, 2025. [Online]. Available: https://arxiv.org/abs/2508.02866

[12] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering (ICSE)*, 2026.

[13] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[14] R. Wei, L. Zhu, H. Wang, R. Yang, J. Guan, S. Ji, Y. Hu, Z. Jiang, and X. Ji, "Model-driven discipline for multi-agent LLMs: Requirement-to-verification generation of traceable system models," arXiv:2607.16708, 2026. [Online]. Available: https://arxiv.org/abs/2607.16708

[15] Y. Yang, N. P. Bhatt, K. Wang, S. Tetteh, Z. Wang, and U. Topcu, "VASO: Formally verifiable self-evolving skills for physical AI agents," arXiv:2606.05395, 2026. [Online]. Available: https://arxiv.org/abs/2606.05395

[16] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Database Theory — ICDT 2001*, ser. LNCS, vol. 1973. Springer, 2001, pp. 316–330.

[17] L. Moreau, B. Clifford, J. Freire, J. Futrelle, Y. Gil, P. Groth, N. Kwasnikowska, S. Miles, P. Missier, J. Myers, B. Plale, Y. Simmhan, E. Stephan, and J. V. den Bussche, "The open provenance model core specification (v1.1)," *Future Generation Computer Systems*, vol. 27, no. 6, pp. 743–756, 2011.

[18] S. Lehnert, "A taxonomy for software change impact analysis," in *Proceedings of the 12th International Workshop on Principles of Software Evolution and the 7th Annual ERCIM Workshop on Software Evolution (IWPSE-EVOL)*, 2011, pp. 41–50.

[19] M. Weiser, "Program slicing," *IEEE Transactions on Software Engineering*, vol. SE-10, no. 4, pp. 352–357, 1984.

[20] B. Meyer, "Applying "design by contract"," *Computer*, vol. 25, no. 10, pp. 40–51, 1992.

[21] D. C. Schmidt, "Model-driven engineering," *Computer*, vol. 39, no. 2, pp. 25–31, 2006.

[22] T. Mens and P. V. Gorp, "A taxonomy of model transformation," *Electronic Notes in Theoretical Computer Science*, vol. 152, pp. 125–142, 2006.

[23] J. Cleland-Huang, O. Gotel, J. H. Hayes, P. Mäder, and A. Zisman, "Software traceability: Trends and future directions," in *Future of Software Engineering (FOSE 2014)*, 2014, pp. 55–69.

[24] A. Egyed, "A scenario-driven approach to trace dependency analysis," *IEEE Transactions on Software Engineering*, vol. 29, no. 2, pp. 116–132, 2003.

[25] C. Nentwich, W. Emmerich, A. Finkelstein, and E. Ellmer, "Flexible consistency checking," *ACM Transactions on Software Engineering and Methodology*, vol. 12, no. 1, pp. 28–63, 2003.

[26] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[27] G. J. Holzmann, "The model checker SPIN," *IEEE Transactions on Software Engineering*, vol. 23, no. 5, pp. 279–295, 1997.

[28] M. Leucker and C. Schallhart, "A brief account of runtime verification," *The Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[29] G. Blair, N. Bencomo, and R. B. France, "Models@run.time," *Computer*, vol. 42, no. 10, pp. 22–27, 2009.

[30] SPIN Project, "Spin—formal verification," Official documentation, version 6.5.2; accessed November 6, 2026. [Online]. Available: https://spinroot.com/spin/whatispin.html

[31] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[32] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[33] MineDojo, "Voyager source repository," GitHub, commit 55e45a880755d0c8c66ca7fb5fe7962ac8974f89, 2023. [Online]. Available: https://github.com/MineDojo/Voyager

[34] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[35] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[36] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.

[37] A. Arcuri and L. Briand, "A practical guide for using statistical tests to assess randomized algorithms in software engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, 2011, pp. 1–10.

[38] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[39] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[40] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[41] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[42] L. Murta, V. Braganholo, F. Chirigati, D. Koop, and J. Freire, "noWorkflow: Capturing and analyzing provenance of scripts," in *Provenance and Annotation of Data and Processes (IPAW 2014)*, ser. LNCS, vol. 8628. Springer, 2014, pp. 71–83.

[43] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium*, 2019, pp. 1393–1410.

[44] T. Stalnaker, N. Wintersgill, O. Chaparro, M. D. Penta, D. M. German, and D. Poshyvanyk, "BOMs Away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, 2024.

[45] W. Gill, A. Anwar, and M. A. Gulzar, "TraceFL: Interpretability-driven debugging in federated learning via neuron provenance," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025, pp. 2264–2276.

[46] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," in *Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2010, pp. 393–407.

[47] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," arXiv:2503.18813, 2025.

[48] M. Costa *et al.*, "Securing AI agents with information-flow control," arXiv:2505.23643, 2025.
[49] P. Buneman, S. Khanna, and W.-C. Tan, "On propagation of deletions and annotations through views," in *Proceedings of the 21st ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, 2002, pp. 150–158.
[50] A. Gupta, I. S. Mumick, and V. S. Subrahmanian, "Maintaining views incrementally," in *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*. ACM, 1993, pp. 157–166.
[51] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance semirings," in *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*. ACM, 2007, pp. 31–40.
[52] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *Proceedings of the 42nd IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 141–159.
[53] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build systems à la carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, pp. 79:1–79:29, 2018.
[54] M. Vartak, H. Subramanyam, W.-E. Lee, S. Viswanathan, S. Husnoo, S. Madden, and M. Zaharia, "ModelDB: A system for machine learning model management," in *Proceedings of the Workshop on Human-In-the-Loop Data Analytics (HILDA)*, 2016.
[55] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.
[56] L. Burgueño, J. Troya, M. Wimmer, and A. Vallecillo, "Static fault localization in model transformations," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 490–506, 2015.
[57] D. B. Abeywickrama, M. Fisher, F. Wheeler, and L. A. Dennis, "Towards continuous assurance with formal verification and assurance cases," arXiv:2511.14805, 2025. [Online]. Available: https://arxiv.org/abs/2511.14805
[58] D. Jackson, "Alloy: A lightweight object modelling notation," *ACM Transactions on Software Engineering and Methodology*, vol. 11, no. 2, pp. 256–290, 2002.
[59] E. Ghassabani, M. Whalen, A. Gacek, and M. Heimdahl, "Inductive validity cores," *IEEE Transactions on Software Engineering*, vol. 47, no. 2, pp. 279–299, 2021.